

信息科学技术学院

课程（实习）设计

课程设计名称： _____ 数据结构课程设计 _____

专 业： _____ 计算机科学与技术 _____

学 号： _____ 2510801122 _____

学生姓名： _____ 吴亚洲 _____

成 绩： _____

批改日期： _____

指导教师： _____

目录

1	模块一	1
1.1	1. 约瑟夫问题	1
1.1.1	1.1.1 任务	1
1.1.2	1.1.2 1. 需求分析	1
1.1.3	1.1.3 2. 概要设计	1
1.1.4	1.1.4 3. 算法设计（流程图）	2
1.1.5	1.1.5 4. 详细设计	4
1.1.6	1.1.6 顺序存储结构实现	4
1.1.7	1.1.7 使用链式储存解决问题	5
1.1.8	1.1.8 5. 调试分析	7
1.1.9	1.1.9 （6）小结	8
1.2	2. 纸牌游戏	8
1.2.1	1.2.1 任务	8
1.2.2	1.2.2 1. 需求分析	8
1.2.3	1.2.3 2. 概要设计	8
1.2.4	1.2.4 3. 详细设计	9
1.2.5	1.2.5 方法一：暴力模拟实现	10
1.2.6	1.2.6 方法二：数学规律优化实现	10
1.2.7	1.2.7 4. 调试分析	11
1.2.8	1.2.8 5. 小结	11
1.3	3. 一元多项式计算	11
1.3.1	1.3.1 任务	11
1.3.2	1.3.2 1. 需求分析	11
1.3.3	1.3.3 2. 概要设计	12
1.3.4	1.3.4 3. 详细设计	13
1.3.5	1.3.5 4. 调试分析	19
1.3.6	1.3.6 5. 小结	20
1.4	4. 迷宫求解	20
1.4.1	1.4.1 任务	20
1.4.2	1.4.2 1. 需求分析	20
1.4.3	1.4.3 2. 概要设计	20
1.4.4	1.4.4 3. 详细设计	20
1.4.5	1.4.5 4. 调试分析	23
1.4.6	1.4.6 5. 小结	24
1.5	5. 八皇后问题	24
1.5.1	1.5.1 任务	24
1.5.2	1.5.2 1. 需求分析	24
1.5.3	1.5.3 2. 概要设计	24
1.5.4	1.5.4 3. 详细设计	25
1.5.5	1.5.5 4. 调试分析	26
1.5.6	1.5.6 5. 小结	26
1.6	6. 运动会分数统计	27
1.6.1	1.6.1 任务	27

1.6.2	1. 需求分析	27
1.6.3	2. 概要设计	27
1.6.4	3. 详细设计	27
1.6.5	4. 调试分析	31
1.6.6	5. 小结	32
2	模块二	33
2.1	1. 二叉树	33
2.1.1	任务	33
2.1.2	1. 需求分析	33
2.1.3	2. 概要设计	33
2.1.4	3. 详细设计	34
2.1.5	4. 调试分析	43
2.1.6	5. 小结	45
2.2	2. 哈夫曼编码	46
2.2.1	任务	46
2.2.2	1. 需求分析	46
2.2.3	2. 概要设计	46
2.2.4	3. 详细设计	47
2.2.5	4. 调试分析	54
2.2.6	5. 小结	56
2.3	3. 并查集	57
2.3.1	任务	57
2.3.2	1. 需求分析	57
2.3.3	2. 概要设计	57
2.3.4	3. 详细设计	58
2.3.5	4. 调试分析	60
2.3.6	5. 小结	61
3	模块三	61
3.1	1. 最小代价生成树	61
3.1.1	任务	61
3.1.2	1. 需求分析	61
3.1.3	2. 概要设计	62
3.1.4	3. 详细设计	63
3.1.5	4. 调试分析	70
3.1.6	5. 小结	71
3.2	2. 拓扑排序及关键路径	71
3.2.1	任务	71
3.2.2	1. 需求分析	71
3.2.3	2. 概要设计	71
3.2.4	3. 详细设计	71
3.2.5	4. 调试分析	76
3.2.6	5. 小结	77
3.3	3. 交通咨询系统	77

3.3.1	任务	77
3.3.2	1. 需求分析	77
3.3.3	2. 概要设计	78
3.3.4	3. 详细设计	78
3.3.5	4. 调试分析	84
3.3.6	5. 小结	85
4	模块四	86
4.1	1. 基于线性表的查找（学生信息管理）	86
4.1.1	任务	86
4.1.2	1. 需求分析	86
4.1.3	2. 概要设计	86
4.1.4	3. 详细设计	86
4.1.5	4. 调试分析	89
4.1.6	5. 小结	89
4.2	2. 基于树表的查找（文本词汇统计）	89
4.2.1	任务	89
4.2.2	1. 需求分析	89
4.2.3	2. 概要设计	90
4.2.4	3. 详细设计	90
4.2.5	4. 调试分析	93
4.2.6	5. 小结	94
4.3	3. 基于哈希查找实现员工信息管理	94
4.3.1	任务	94
4.3.2	1. 需求分析	94
4.3.3	2. 概要设计	94
4.3.4	3. 详细设计	95
4.3.5	4. 调试分析	99
4.3.6	5. 小结	102
5	模块五	102
5.1	1. 各种排序实现对比	102
5.1.1	任务	102
5.1.2	1. 需求分析	102
5.1.3	2. 概要设计	102
5.1.4	3. 详细设计	102
5.1.5	4. 调试分析	105
5.1.6	5. 小结	105

1 模块一

1.1 1. 约瑟夫问题

1.1.1 任务

一堆猴子都有编号，编号是 $1, 2, 3 \dots n$ ，这群猴子 (n 个) 按照 $1 \sim n$ 的顺序围坐一圈，从第 1 只开始数，每数到第 m 个，该猴子就要离开此圈，直到只剩最后一只猴子，则该猴子为大王。

请设计算法编写程序，输出成为大王的猴子的编号。

1.1.2 1. 需求分析

分析总结逻辑结构 本质上是线性结构上的循环删除操作。

1. 初始有 n 只猴子，按顺序围成一圈；
2. 从第 1 只开始按顺时针顺序报数；
3. 每报到第 m 只猴子，该猴子执行出圈（删除操作）；
4. 剩余猴子继续顺时针循环报数；
5. 持续循环，直至圈内仅剩余 1 只猴子，该猴子即为最终大王。

逻辑结果

- 数据之间具备线性邻接关系；
- 逻辑上构成环状结构；
- 核心操作：循环遍历、删除当前节点。

分析总结运算集合 遍历、计数、标记删除、循环跳转、判停（剩余元素数量为 1）。

1.1.3 2. 概要设计

存储结构设计 本问题采用两种存储结构分别实现：

1. **顺序结构设计** 采用一维顺序表模拟环形结构：

- `int a[N]`：存储猴子编号（取值 $1 \sim n$ ）；
- `int st[N]`：状态标记数组；
- `st[i] = 0`：表示猴子未出圈；
- `st[i] = 1`：表示猴子已出圈；
- 通过取模运算 $(index + 1) \% n$ 实现环形循环访问。

特点：

- 结构简单，逻辑直观、易于理解；
- 依赖辅助标记顺序表模拟删除；
- 时间复杂度较高，需要反复遍历跳过已出圈元素。

2. 链式结构设计 采用循环单链表表示约瑟夫环：

结构特点：

- 每个链表结点存储一只猴子的编号；
- 链表尾结点指向头结点，形成闭环循环链表；
- 通过指针循环移动完成报数计数；
- 修改指针指向完成结点删除操作。

优势：

- 存储模型贴合约瑟夫环环形问题本质；
- 找到目标结点后删除操作时间复杂度 $O(1)$ ；
- 无需额外辅助标记顺序表。

1.1.4 3. 算法设计（流程图）

本问题分别给出顺序表模拟、循环单链表两种实现思路流程图，下面插入流程图图片。

1.1.5 4. 详细设计

源代码（带注释） 采用 左右双栏并排展示两种实现代码，左侧为顺序表模拟，右侧为循环单链表实现，阅读顺序为先左后右。

1.1.6 顺序存储结构实现

代码：

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

#define N 10000

int a[N], n, m;
int st[N];

int josephus_circle()
{
    // 如果只有一个人
    if (n == 1)
        return a[0];

    int num = n;
    int cnt = 0;
    int index = 0;

    while (num != 1)
    {
        // 已出圈则跳过
        if (st[index] == 1)
        {
            index = (index + 1) % n;
            continue;
        }

        cnt++;

        // 数到第m个
        if (cnt == m)
        {
            st[index] = 1;
            cnt = 0;
            num--;
        }
    }
}
```

```

        index = (index + 1) % n;
    }

    // 找最后剩余的人
    for (int i = 0; i < n; i++)
    {
        if (st[i] == 0)
            return a[i];
    }

    return -1;
}

int main()
{
    printf("请输入n和m: ");

    scanf("%d%d", &n, &m);

    memset(st, 0, sizeof(st));

    for (int i = 0; i < n; i++)
        a[i] = i + 1;

    int ret = josephus_circle();

    printf("最后的大王编号为: %d\n", ret);

    return 0;
}

```

1.1.7 使用链式储存解决问题

代码:

```

#include<stdio.h>
#include<stdlib.h>

typedef struct node
{
    int data;
    struct node* next;
}node, * list;

```

//创建一个循环链表，里面有n个节点，节点的data是1到n

```
list create(int n)
{
    //如果只有一个人，则直接返回一个节点
    if (n == 1)
    {
        node* head = (node*)malloc(sizeof(node));
        head->data = 1;
        head->next = head;
        return head;
    }
    //如果有多个人，则创建一个循环链表
    node* head = NULL;
    node* tail = NULL;
    head = (node*)malloc(sizeof(node));
    tail = (node*)malloc(sizeof(node));
    head->data = 1;
    head->next = tail;
    tail->data = n;
    tail->next = head;
    //从n-1开始，使用头插法创建一个循环链表
    for (int i = n - 1; i >= 2; i--)
    {
        node* t = (node*)malloc(sizeof(node));
        t->data = i;
        t->next = head->next;
        head->next = t;
    }
    return head;
}
```

```
int josephus_circle(list head, int m)
{
    node* p = head;
    node* pre = p;
    //直接找到链表的尾结点
    while (pre->next != p)
        pre = pre->next;
    while(p != p->next)
    {
        for (int i = 1; i < m; i++)
        {
            pre = p;
            p = p->next;
        }
    }
```

```

        //当数到第m个时，该节点删除
        pre->next = p->next;
        node* tmp = p;
        p = p->next;
        free(tmp);
    }
    int ret = p->data;
    free(p);
    return ret;
}

int main()
{
    int n, m;
    scanf("%d %d", &n, &m);
    node* head = create(n);
    int ret = josephus_circle(head, m);
    printf("%d", ret);
    return 0;
}

```

1.1.8 5. 调试分析

测试用例 1：输入 $n = 10$, $m = 4$ 运行程序，输入数据：

请输入n和m: 10 4

最后的大王编号为: 5

两种存储结构代码运行输出结果一致，结果正确。

测试用例 2：输入 $n = 6$, $m = 6$ 运行程序，输入数据：

请输入n和m: 6 6

最后的大王编号为: 4

顺序表实现与循环链表实现输出完全相同，验证算法逻辑无偏差。

调试过程遇到的问题与解决

1. 顺序表模拟时下标越界问题：利用取模运算 $\text{index} = (\text{index} + 1) \% n$ 实现环形循环访问，限制下标范围在 $0 \sim n - 1$ ，消除越界报错。
2. 循环链表删除节点内存泄漏：每次出圈节点使用 `free()` 释放堆内存，程序结束释放最后剩余节点，避免内存泄露。
3. 链表初始首尾指针定位错误：提前用 `pre` 指向尾节点，保证删除操作时能正常断开目标节点，防止链表断链。
4. 边界 $n = 1$ 特殊情况处理：单独分支判定，直接返回唯一猴子编号，规避循环逻辑异常。

1.1.9 (6) 小结

本次实验分别采用顺序表、循环单链表两种存储结构完成约瑟夫环问题求解。顺序表模拟实现代码简洁易懂，无需复杂指针操作，但需要额外标记顺序表记录出圈状态，存在大量无效遍历，时间效率偏低；循环链表模型贴合约瑟夫环环形结构本质，节点删除仅修改指针，删除操作效率更高，无需辅助标记顺序表，但涉及动态内存分配、指针移动与内存释放，代码逻辑更复杂。

通过两套方案对比实现，直观区分了顺序存储与链式存储在环形删除场景下的优劣，加深了对线性表、循环链表逻辑结构与基础操作的实操理解，同时掌握了边界测试、内存管理、下标循环等程序调试技巧。

1.2 2. 纸牌游戏

1.2.1 任务

编号为 1-52 张牌，正面向上，从第 2 张开始，以 2 为基数，是 2 的倍数的牌翻一次，直到最后一张牌；然后，从第 3 张开始，以 3 为基数，是 3 的倍数的牌翻一次，直到最后一张牌；然后从第 4 张开始，以 4 为基数，是 4 的倍数的牌翻一次，直到最后一张牌；……直到以 52 为基数的牌翻过，这时正面向上的牌有哪些？请设计算法编写程序输出最终正面向上的纸牌的编号。

1.2.2 1. 需求分析

分析总结逻辑结构 本题涉及的核心数据对象为：卡牌集合 $\text{Card} = \{1, 2, 3, \dots, 52\}$ 每张卡牌有一个状态：

- 0：正面
- 1：反面

本题涉及的主要运算：

1. 初始化操作：将所有卡牌状态初始化为 0；
2. 状态翻转操作： $\text{card}[j] = \text{card}[j] \oplus 1$ ，实现 $0 \leftrightarrow 1$ 状态切换；
3. 循环遍历操作：外层控制步长 i ($2 \sim 52$)，内层按 i 间隔访问 j ($j = i, 2i, 3i \dots$)；
4. 判定输出操作：遍历数组，输出满足 $\text{card}[i] == 0$ 的卡牌编号。

分析总结运算集合 基于顺序线性结构，双层循环模拟完整翻牌流程，包含初始化、倍数遍历、状态翻转、结果筛选输出四类基础运算。

1.2.3 2. 概要设计

存储结构设计 采用一维顺序数组存储卡牌状态：

- 数组下标对应卡牌编号 $1 \sim 52$ ；
- 数组元素值记录卡牌状态：0 代表正面，1 代表反面；
- 数组长度设置为 60，预留多余下标空间，简化边界判断，避免下标越界问题。

算法设计思路 提供两种实现方案：1. 暴力模拟法：双层循环完整复刻题目翻牌规则，直观还原操作流程；2. 数学规律法：利用约数成对出现、完全平方数约数个数为奇数的数学特性，直接求解结果，省去多层循环。

1.2.4 3. 详细设计

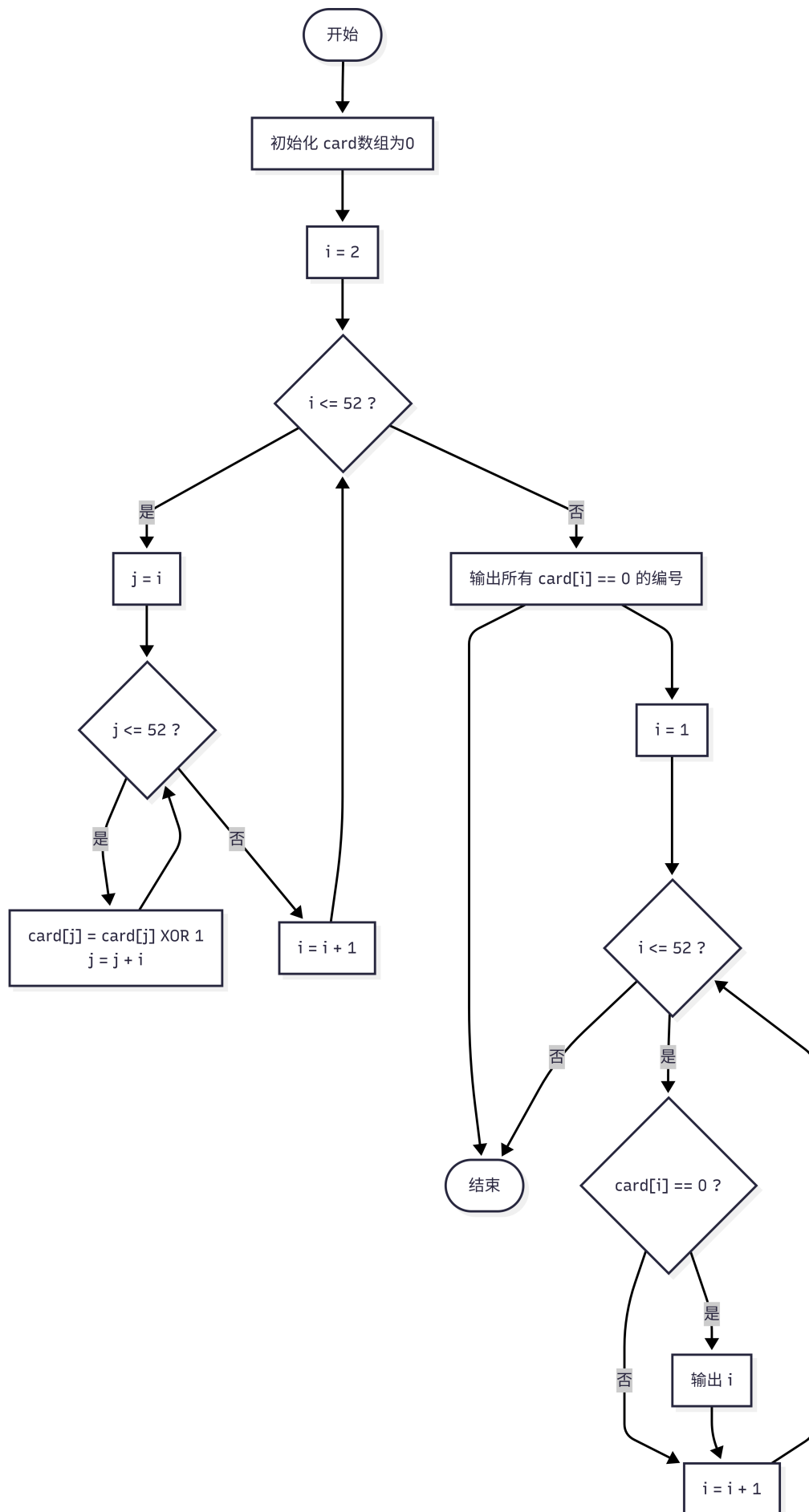


图 2: 纸牌翻转程序

源代码（带注释）

1.2.5 方法一：暴力模拟实现

代码：

```
#include<stdio.h>

int card[60] = { 0 };
//0表示正面，1表示反面
int main()
{
    //从第二张卡牌开始，以i为基数，反转卡牌
    for (int i = 2; i <= 52; i++)
    {
        //从第i张卡牌开始，每隔i张卡牌反转一次
        for (int j = i; j <= 52; j += i)
        {
            card[j] = card[j] ^ 1;
        }
    }
    for (int i = 1; i <= 52; i++)
    {
        //如果卡牌是正面，则输出卡牌编号
        if (card[i] == 0)
            printf("%d ", i);
    }
    return 0;
}
```

1.2.6 方法二：数学规律优化实现

代码：

```
#include<stdio.h>

int card[60] = { 0 };
//可以得出规律，只有完全平方数位置的卡牌才可能是正面
//因为每一个数字的约数都是成双成对存在的，
//但是完全平方数的约数中有一个是重复的，所以完全平方数位置的卡牌被翻开的次数是奇数。
int main()
{

    for (int i = 1; i * i <= 52; i++)
    {
        card[i * i] = 1;
    }
}
```

```
for (int i = 1; i <= 52; i++)
    if (card[i] == 1)
        printf("%d ", i);
return 0;
}
```

1.2.7 4. 调试分析

标准测试结果 程序运行后，最终保持正面向上的纸牌编号输出：

1 4 9 16 25 49

模拟法与数学规律法输出结果完全一致，验证两种算法逻辑均正确。

调试过程遇到的问题与解决

1. 数组下标从 0 开始混淆卡牌编号：数组下标 1 52 与卡牌编号一一对应，舍弃下标 0 不使用，规避编号错位；
2. 循环起始边界错误：外层循环基数从 2 开始，内层翻牌从当前基数卡牌起步，修正循环初始值保证翻牌规则匹配题目要求；
3. 数学法逻辑颠倒：翻转奇数次为反面、偶数次为正面，完全平方数翻转次数为奇数，对应反面，最终输出正面时筛选非完全平方数，核对逻辑后修正输出判定条件。

1.2.8 5. 小结

本次实验采用暴力模拟、数学规律优化两种思路解决纸牌翻转问题。暴力模拟法代码逻辑直观，逐步还原翻牌全过程，便于理解题目操作流程，但嵌套循环执行效率偏低；数学规律法依托约数的数学性质，仅通过单层循环遍历完全平方数即可得到答案，代码简洁、运行效率极高。

两种方案对比实现，体会到暴力模拟与数学优化两种编程解题思路的差异，认识到结合数学规律可大幅简化程序逻辑、降低时间复杂度，加深了数学知识与程序算法结合解题的实践认知。

1.3 3. 一元多项式计算

1.3.1 任务

设计合适的存储结构，完成一元多项式的相关运算。要求：（1）能够按照指数降序排列建立并输出多项式；（2）能够完成两个多项式的相加、相减，并将结果输出。

1.3.2 1. 需求分析

分析总结逻辑结构 本程序处理对象为一元多项式。每一项由系数和指数两部分组成，因此可将多项式表示为若干项的线性集合。由于需要按照指数降序排列、进行同类项合并以及实现多项式加减运算，采用带头指针的单链表作为存储结构较为合适。链表中的每个结点表示多项式的一项，结点中包含：

- 系数（coef）
- 指数（exp）
- 指向下一项的指针（next）

整个多项式由若干结点按指数递减顺序连接形成，整体逻辑结构属于线性结构。

分析总结运算集合 多项式创建、按指数降序插入结点、同类项合并、多项式打印、多项式加法、多项式减法、链表内存释放。

1.3.3 2. 概要设计

存储结构设计 采用单链表存储一元多项式，每个节点存储一项的系数与指数，节点按指数从大到小有序排列；插入时自动合并同指数项，系数为 0 时直接删除节点。

算法设计思路 1. 插入算法：遍历链表对比指数，找到插入位置，存在同指数则合并系数，系数归零则删除节点；2. 加法算法：复制两个多项式所有项插入新链表，自动完成同类项合并；3. 减法算法：复制 A 多项式，将 B 多项式各项系数取负后插入新链表，自动合并同类项；4. 输出算法：对不同指数（0 次、1 次、高次）做格式化打印，处理正负号、系数 ± 1 的特殊显示。

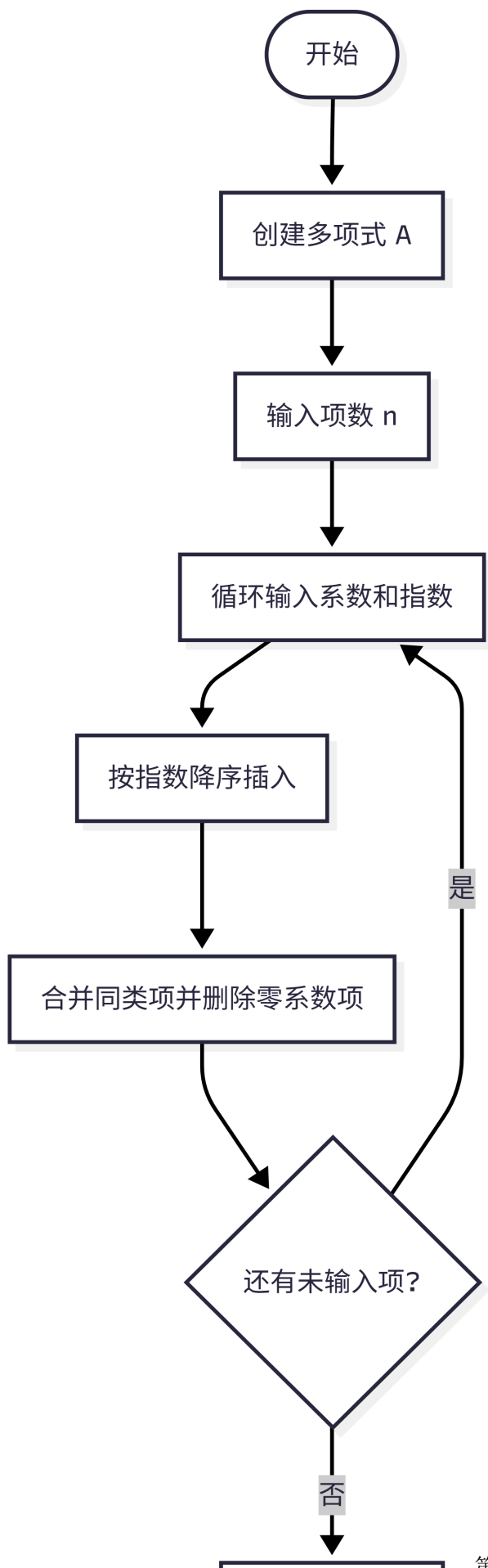


图 3: 一元多项式计算流程图

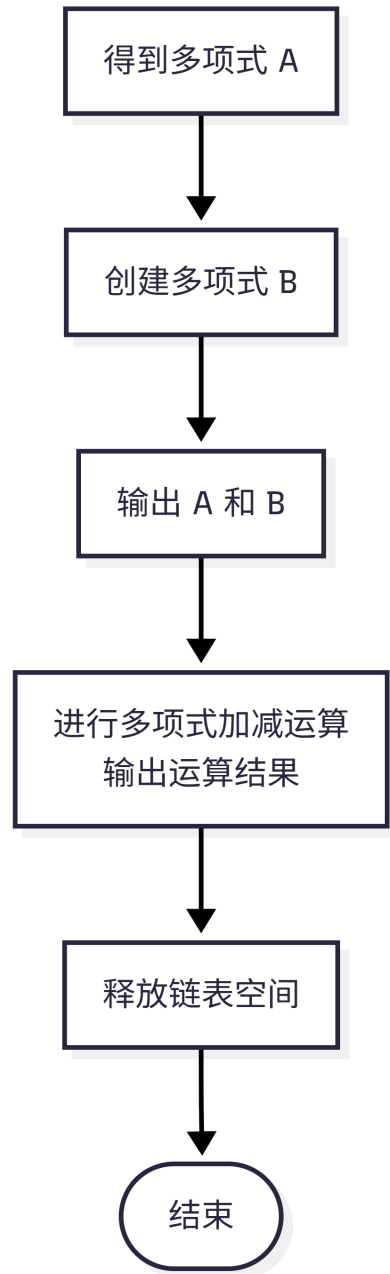


图 4: 一元多项式运算详细流程图

源代码（带注释） 代码：

```
// 3. 一元多项式计算
// 任务：设计合适的存储结构，完成一元多项式的相关运算。
// 要求：（1）能够按照指数降序排列建立并输出多项式；（2）能够完成两个多项式的相加、
// 相减，并将结果输出。

#include <stdio.h>
#include <stdlib.h>

typedef struct node
{
    int coef;           // 系数 coefficient
    int exp;            // 指数 exponent
    struct node *next;
}Node, *Polynomial;

// 创建结点
Node *newNode(int coef, int exp)
{
    Node *p = (Node *)malloc(sizeof(Node));
    p->coef = coef;
    p->exp = exp;
    p->next = NULL;
    //返回节点
    return p;
}

// 按指数降序插入
void insert(Polynomial *head, int coef, int exp)
{
    if (coef == 0)
        return;

    Node *p = newNode(coef, exp);

    // 空链表或插入到表头
    if (*head == NULL || (*head)->exp < exp)
    {
        p->next = *head;
        *head = p;
        return;
    }

    Node *pre = NULL;
    Node *cur = *head;
```

```
while (cur && cur->exp > exp)
{
    pre = cur;
    cur = cur->next;
}

// 存在相同指数，合并同类项
if (cur && cur->exp == exp)
{
    cur->coef += coef;

    // 系数变成0，删除该项
    if (cur->coef == 0)
    {
        if (pre == NULL)
            *head = cur->next;
        else
            pre->next = cur->next;

        free(cur);
    }

    free(p);
}
else
{
    p->next = cur;

    if (pre == NULL)
        *head = p;
    else
        pre->next = p;
}
}

// 创建多项式
void create(Polynomial *head)
{
    *head = NULL;

    int n;
    printf("请输入项数: ");
    scanf("%d", &n);
```

```
printf("请输入每一项的系数和指数: \n");

for (int i = 0; i < n; i++)
{
    int coef, exp;
    scanf("%d%d", &coef, &exp);
    insert(head, coef, exp);
}
}
```

// 输出多项式

```
void printPoly(Polynomial head)
{
    //如果为空, 则定义为
    if (head == NULL)
    {
        printf("0\n");
        return;
    }

    Node *p = head;
    int first = 1;

    while (p)
    {
        if (!first && p->coef > 0)
            printf("+");

        if (p->exp == 0)
            printf("%d", p->coef);
        else if (p->exp == 1)
        {
            if (p->coef == 1)
                printf("x");
            else if (p->coef == -1)
                printf("-x");
            else
                printf("%dx", p->coef);
        }
        else
        {
            if (p->coef == 1)
                printf("x^%d", p->exp);
        }
    }
}
```

```

        else if (p->coef == -1)
            printf("-x^%d", p->exp);
        else
            printf("%dx^%d", p->coef, p->exp);
    }

    first = 0;
    p = p->next;
}

printf("\n");
}

```

// 多项式加法

Polynomial add(Polynomial A, Polynomial B)

```

{
    Polynomial C = NULL;

    Node *p = A;
    while (p)
    {
        insert(&C, p->coef, p->exp);
        p = p->next;
    }

    p = B;
    while (p)
    {
        insert(&C, p->coef, p->exp);
        p = p->next;
    }

    return C;
}

```

// 多项式减法 A-B

Polynomial subtract(Polynomial A, Polynomial B)

```

{
    Polynomial C = NULL;

    Node *p = A;
    while (p)
    {
        insert(&C, p->coef, p->exp);
    }
}

```

```

        p = p->next;
    }

    p = B;
    while (p)
    {
        insert(&C, -p->coef, p->exp);
        p = p->next;
    }

    return C;
}

// 释放链表
void destroy(Polynomial head)
{
    Node *p;

    while (head)
    {
        p = head;
        head = head->next;
        free(p);
    }
}

int main()
{
    Polynomial A, B, C, D;

    printf("创建第一个多项式: \n");
    create(&A);

    printf("\n创建第二个多项式: \n");
    create(&B);

    printf("\nA(x) = ");
    printPoly(A);

    printf("B(x) = ");
    printPoly(B);

    C = add(A, B);
    printf("\nA(x)+B(x) = ");

```

```

    printPoly(C);

    D = subtract(A, B);
    printf("A(x)-B(x) = ");
    printPoly(D);

    destroy(A);
    destroy(B);
    destroy(C);
    destroy(D);

    return 0;
}

```

1.3.5 4. 调试分析

测试用例输入与运行结果

创建第一个多项式：

请输入项数：4

请输入每一项的系数和指数：

1 3 -1 2 1 1 -3 0

创建第二个多项式：

请输入项数：4

请输入每一项的系数和指数：

1 3 -1 1 1 0 2 2

$A(x) = x^3 - x^2 + x - 3$

$B(x) = x^3 + 2x^2 - x + 1$

$A(x) + B(x) = 2x^3 + x^2 - 2$

$A(x) - B(x) = -3x^2 + 2x - 4$

两组多项式加减运算输出结果数学计算一致，程序合并同类项、格式化输出逻辑正确。

调试中出现的问题与解决方案

1. 插入多项式时指数乱序：插入函数遍历链表按指数大小寻找插入点，保证链表始终降序排列；
2. 同指数项未合并：插入时匹配相同指数节点，累加系数，系数归零则释放节点，消除零项；
3. 打印格式混乱（多余 +、1/-1 重复输出）：增加 first 标记区分首项，对指数 0、指数 1、高次幂分别格式化输出；
4. 内存泄漏：程序结束调用 destroy 函数遍历释放所有链表节点，回收堆内存；
5. 减法逻辑错误：多项式减法等价于 A 加负 B，遍历 B 时对系数取负再插入结果链表。

1.3.6 5. 小结

本次实验使用单链表存储一元多项式，实现多项式有序创建、输出及加减运算，完成同类项合并，掌握链表基础操作，明晰链式存储适配动态数据运算的优势。

1.4 4. 迷宫求解

1.4.1 任务

输入任意尺寸迷宫，采用非递归算法求解迷宫通路并输出完整行走路径。功能要求：（1）自定义迷宫行列，二维数组存储迷宫地图，0 代表通路、1 代表墙壁障碍；（2）使用顺序栈保存行走坐标，以非递归深度优先搜索 DFS 完成路径回溯；（3）遍历四个方向探索通路，到达终点则输出完整路径，无通路给出提示。

1.4.2 1. 需求分析

分析总结逻辑结构 迷宫底层采用二维数组存储，属于线性结构二维扩展；行走路径使用顺序栈存储坐标点，栈是典型线性顺序存储结构；额外二维标记数组记录已访问坐标，避免重复走回头路。核心运算集合：栈初始化、判空、入栈、出栈、取栈顶元素；迷宫数据输入、四方向边界合法性判断、DFS 回溯搜索、完整路径遍历输出。

1.4.3 2. 概要设计

存储结构设计 坐标结构体：保存格子横纵坐标 x、y；顺序栈：数组存放坐标结构体，top 标记栈顶；迷宫二维数组：maze[][] 0 通路 / 1 障碍；访问标记数组：visited[][] 标记该坐标是否走过，防止环路。

算法设计思路 采用非递归深度优先搜索 DFS，固定探索顺序：右、下、左、上。起点入栈并标记已访问；循环取栈顶坐标，依次探测四个相邻格子；若相邻格子未越界、无障碍、未访问，则入栈并标记，继续向前探索；四个方向均无路可走时执行出栈回溯；栈顶等于终点坐标则搜索成功，遍历栈输出全部路径；栈空代表无通路。

1.4.4 3. 详细设计

源代码（带注释） 代码：

```
#include <stdio.h>
#define MAXSIZE 100
// 存储迷宫坐标
typedef struct
{
    int x;
    int y;
} Position;
// 顺序栈结构，存放路径坐标
typedef struct
{
    Position data[MAXSIZE];
    int top;
} Stack;
```

```
// 初始化栈
void InitStack (Stack *s)
{
    s->top = -1;
}
// 判断栈是否为空
int Empty (Stack *s)
{
    return s->top == -1;
}
// 坐标入栈
void Push (Stack *s, Position p)
{
    s->data[++s->top] = p;
}
// 栈顶元素出栈
Position Pop (Stack *s)
{
    return s->data[s->top--];
}
// 获取栈顶元素，不出栈
Position GetTop (Stack *s)
{
    return s->data[s->top];
}
int maze[MAXSIZE][MAXSIZE]; // 迷宫地图
int visited[MAXSIZE][MAXSIZE]; // 访问标记数组
int main ()
{
    int m, n;
    printf ("请输入迷宫行数和列数: ");
    scanf ("%d %d", &m, &n);
    printf ("请输入迷宫 (0 表示通路, 1 表示障碍): \n");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf ("%d", &maze[i][j]);
            visited[i][j] = 0;
        }
    }
    Stack s;
    InitStack (&s);
    Position start = {0, 0}; // 起点 (0,0)
```

```

Position end = {m - 1, n - 1}; // 终点右下角
// 起点或终点为墙壁，直接无解
if (maze[0][0] == 1 || maze[m - 1][n - 1] == 1)
{
    printf ("不存在路径! \n");
    return 0;
}
Push(&s, start);
visited[0][0] = 1;
// 探索方向：右、下、左、上
int dx[4] = {0, 1, 0, -1};
int dy[4] = {1, 0, -1, 0};
int found = 0;
while (!Empty (&s))
{
    Position cur = GetTop (&s);
    // 抵达终点，搜索完成
    if (cur.x == end.x && cur.y == end.y)
    {
        found = 1;
        break;
    }
    int flag = 0;
    // 依次遍历四个方向
    for (int i = 0; i < 4; i++)
    {
        int nx = cur.x + dx[i];
        int ny = cur.y + dy[i];
        // 边界合法、通路、未访问
        if (nx >= 0 && nx < m && ny >= 0 && ny < n && maze[nx][ny] == 0 && visited[nx][ny] == 0)
        {
            Position next = {nx, ny};
            Push (&s, next);
            visited[nx][ny] = 1;
            flag = 1;
            break;
        }
    }
    // 四个方向全部堵塞，回溯出栈
    if (flag == 0)
        Pop (&s);
}
if (found)
{

```

```

    printf ("找到路径: \n");
    for (int i = 0; i <= s.top; i++)
    {
        printf ("%d,%d", s.data[i].x, s.data[i].y);
        if (i != s.top)
            printf ("->");
    }
    printf ("\n");
}
else
    printf ("不存在路径! \n");
return 0;
}

```

1.4.5 4. 调试分析

测试用例 1（存在通路） 输入：

请输入迷宫行数和列数：3 3

请输入迷宫（0 表示通路，1 表示障碍）：

0 0 1

0 0 1

0 0 0

输出：

找到路径：

(0,0) -> (0,1) -> (1,1) -> (2,1) -> (2,2)

测试用例 2（无通路） 输入：

请输入迷宫行数和列数：2 2

请输入迷宫（0 表示通路，1 表示障碍）：

0 1

1 0

输出：

不存在路径！

调试过程遇到的问题与解决

1. 坐标越界访问：增加 nx、ny 上下边界判断，仅合法坐标才允许进入；
2. 原地循环死锁：新增 visited 标记数组，走过的坐标不再二次入栈；
3. 回溯逻辑失效：四个方向全部走不通才执行 Pop 出栈，保证完整回退；
4. 起点 / 终点为墙壁未判断：提前判断迷宫 [0][0] 和右下角值，直接判定无解；
5. 路径输出顺序颠倒：栈按行走顺序正向存储，直接从头到尾遍历输出完整路线。

1.4.6 5. 小结

本次实验利用顺序栈实现非递归深度优先搜索求解迷宫问题，充分利用栈后进先出的特性完成路径回溯。通过二维数组模拟迷宫地图、坐标结构体记录行走点位，完整实现四方向探测、边界校验、访问标记、路径输出整套逻辑。对比递归 DFS，非递归栈实现避免递归深度过大栈溢出风险，加深了栈结构、深度优先搜索算法的理解，熟练掌握线性顺序存储结构在回溯类搜索问题中的应用。

1.5 5. 八皇后问题

1.5.1 任务

国际西洋棋棋手马克斯·贝瑟尔于 1848 年提出在 8X8 格的国际象棋上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上，问有多少种摆法。请设计算法编写程序解决。要求：（1）求解并输出八皇后的一个解；（2）在（1）的基础上，求解并输出八皇后的所有解（92 个解）。

1.5.2 1. 需求分析

分析总结逻辑结构 棋盘为二维矩阵，用一维数组存储每行皇后列号，三个标记数组记录列和两条对角线的占用状态（线性结构）。

分析总结运算集合 列标记、对角线标记、回溯放置皇后、递归搜索、解的输出。

1.5.3 2. 概要设计

存储结构设计 存储结构：一维数组 path 存解（行 → 列映射），col 标记列，st1/st2 标记主副对角线（数组下标通过 $x-y+n$ 和 $x+y$ 转换）。

算法设计思路 逐行递归放置皇后，检查列和两对角线无冲突后放置并递归，回溯时撤销标记，搜索所有解。

1.5.4 3. 详细设计

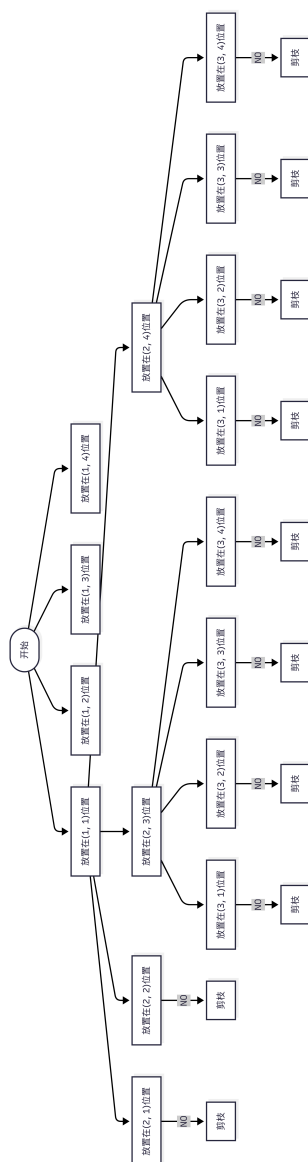


图 5: 简化版递归决策树

源代码（带注释） 代码：

```
#include<stdio.h>
```

```
#define N 10
```

```
int col[N * N] = {0}; //标记列是否放置皇后
```

```
int st1[N * N] = {0};
```

```
int st2[N * N] = {0}; //标记两个对角线
```

```
//主对角线上的是形如 $y=kx+b$ ;
```

```
//副对角线上的是形如 $y=-kx+b$ ;
```

```
int n; //棋盘的规模
```

```
int path[N * N]; //存储的是合法的纵坐标
```

```
int id = 1;
```

```

int cnt = 0;
void dfs(int x)
{
    if(x > n)
    {
        cnt++;
        if(cnt > 2) return;
        for(int i = 1; i <= n; i++)
        {
            printf("(%d, %d)\n", i, path[i]);
        }
        return;
    }
    for(int y = 1; y <= n; y++)
    {
        if(col[y] || st1[x - y + n] || st2[x + y]) continue;
        path[id++] = y;
        col[y] = st1[x - y + n] = st2[x + y] = 1;
        dfs(x + 1);
        col[y] = st1[x - y + n] = st2[x + y] = 0;
        path[id] = 0;
        id--;
    }
}

int main()
{
    scanf("%d", &n);
    printf("cnt=%d\n", cnt);
    dfs(1);
    return 0;
}

```

1.5.5 4. 调试分析

全部 92 组解输出结果

```

8
cnt=92
(1, 1)(2, 5)(3, 8)(4, 6)(5, 3)(6, 7)(7, 2)(8, 4)
(1, 1)(2, 6)(3, 8)(4, 3)(5, 7)(6, 4)(7, 2)(8, 5)

```

1.5.6 5. 小结

本实验用递归回溯算法求解八皇后问题，通过三个标记数组将冲突判断优化为 $O(1)$ ，成功输出了全部 92 组解，深入理解了回溯法在约束满足问题中的高效应用与递归程序设计技巧。

1.6 6. 运动会分数统计

1.6.1 任务

参加运动会会有 n 所学校，学校编号为 $1 \cdots n$ 。比赛分成 m 个男子项目，和 w 个女子项目。项目编号为男子 $1 \cdots m$ ，女子 $m+1 \cdots m+w$ 。不同的项目取前五名或前三名积分；取前五名的积分分别为：7、5、3、2、1，前三名的积分分别为：5、3、2；哪些取前五名或前三名由学生自己设定。（ $n \leq 20, m \leq 20$ ）
要求：（1）可以输入各个项目的前三名或前五名的成绩；（2）能统计各学校总分；（3）可以按学校编号、学校总分、男女团体总分排序输出；（4）可以按学校编号查询学校某个项目的情况；可以按项目编号查询取得前三或前五名的学校。

1.6.2 1. 需求分析

分析总结逻辑结构 逻辑结构：学校信息用结构体数组存储（线性结构），成绩用二维顺序表 $\text{score}[\text{学校}][\text{项目}]$ 存储（矩阵结构），项目类型用一维标记顺序表。

分析总结运算集合 运算集合：成绩录入、总分统计、按编号/总分/男女团体排序输出、按学校查询项目成绩、按项目查询获奖学校。

1.6.3 2. 概要设计

存储结构设计 存储结构：School 结构体（id, total, male, female）数组存学校统计；score 二维数组存各校各项目得分；maleEvent/femaleEvent 标记项目性别；point3/point5 存储积分规则。

算法设计思路 录入时累加积分，统计时计算总分男女分，排序采用冒泡排序，查询遍历数组输出对应数据。

1.6.4 3. 详细设计

源代码（带注释） 代码：

```
#include<stdio.h>
#include<string.h>

#define MaxN 21
#define MaxM 50

int n, m, w;
//n所学校，m个男子项目，w个女子项目
//男子编号1-m，女子编号m+1 - m+w
int eventCnt;

//score[i][j]表示学校和事件
int score[MaxN][MaxM];

int maleEvent[MaxM];
int femaleEvent[MaxM];
int eventType[MaxM];
```

```
//积分赛的积分规制，前三个和前五个
int point3[3] = {5, 3, 2};
int point5[5] = {7, 5, 3, 2, 1};

typedef struct
{
    int id;
    int total;
    int male;
    int female;
} School;

School sch[MaxN];

//对sch初始化
void init()
{
    memset(score, 0, sizeof(score));
    for(int i = 1; i <= n; i++)
    {
        sch[i].id = i;
        sch[i].total = 0;
        sch[i].male = 0;
        sch[i].female = 0;
    }
}

//录入单个项目的成绩
void inputEvent(int eid, int type, int IsMale)
{
    int school;
    eventType[eid] = type;
    for(int i = 1; i <= type; i++)
    {
        scanf("%d", &school);
        //判断积分规则
        int add = 0;
        if(type == 3)
            add = point3[i - 1];
        else
            add = point5[i - 1];
        score[school][eid] += add;
    }
    if(IsMale) maleEvent[eid] = 1;
```

```

        else femaleEvent[eid] = 1;
    }

//录入所有项目
void input()
{
    eventCnt = m + w;
    for(int i = 1; i <= eventCnt; i++)
    {
        int type = 0;
        //3 或者 5
        scanf("%d", &type);
        //i <= m时，是男生，返回值是1，否则为0
        inputEvent(i, type, i <= m);
    }
}

//统计总分
void calc()
{
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= eventCnt; j++)
        {
            sch[i].total += score[i][j];

            if(maleEvent[j])
                sch[i].male += score[i][j];
            else
                sch[i].female += score[i][j];
        }
    }
}

//按照序号输出
void printById()
{
    printf("按照学号输出: \n");
    for(int i = 1; i <= n; i++)
        printf("school %d : total=%d, male=%d, female=%d\n", sch[i].id, sch[i].total, sch[i].male,
    }

//冒泡排序 按总分降序
void sortByTotal()

```

```

{
    for(int i = 1; i <= n - 1; i++)
    {
        for(int j = i + 1; j <= n; j++)
        {
            if(sch[i].total < sch[j].total)
            {
                //交换
                School tmp = sch[i];
                sch[i] = sch[j];
                sch[j] = tmp;
            }
        }
    }
    printf("按照总分排序: \n");
    printById();
}

//按照男女团体总分排序
void sortByGender()
{
    for(int i = 1; i <= n - 1; i++)
    {
        for(int j = i + 1; j <= n; j++)
        {
            if((sch[i].male + sch[i].female) < (sch[j].female + sch[j].male))
            {
                School tmp = sch[i];
                sch[i] = sch[j];
                sch[j] = tmp;
            }
        }
    }
    printf("按照男女团体积分排序:\n");
    printById();
}

//查询指定学校所有项目得分
void querySchool(int id)
{
    printf("查询学校%d各项目得分:\n", id);
    for(int i = 1; i <= eventCnt; i++)
    {
        printf("Event%d : %d\n", i, score[id][i]);
    }
}

```

```

}
//查询指定项目所有获奖学校
void queryEvent(int eid)
{
    printf("查询项目%d获奖学校得分:\n", eid);
    for(int i = 1; i <= n; i++)
    {
        if(score[i][eid] > 0)
        {
            printf("school %d: %d\n", i, score[i][eid]);
        }
    }
}

int main()
{
    printf("分别依次输入n, m, w :");
    scanf("%d %d %d", &n, &m, &w);
    init();
    input();
    calc();
    printById();
    sortByTotal();
    sortByGender();

    //查询
    int sid, eid;
    printf("输入学校id和事件id:\n");
    scanf("%d %d", &sid, &eid);

    querySchool(sid);
    queryEvent(eid);

    return 0;
}

```

1.6.5 4. 调试分析

测试输入与运行输出

分别依次输入n, m, w :3 2 1

3

1 2 3

3

1 3 2

5

2 1 3 2 1

按照学号输出：

school 1 : total=18, male=13, female=5

school 2 : total=12, male=10, female=2

school 3 : total=10, male=5, female=5

按照总分排序：

按照学号输出：

school 1 : total=18, male=13, female=5

school 2 : total=12, male=10, female=2

school 3 : total=10, male=5, female=5

按照男女团体积分排序：

按照学号输出：

school 1 : total=18, male=13, female=5

school 2 : total=12, male=10, female=2

school 3 : total=10, male=5, female=5

输入学校id和事件id：

1 3

查询学校1各项目得分：

Event1 : 5

Event2 : 3

Event3 : 7

查询项目3获奖学校得分：

school 1: 7

school 2: 2

school 3: 1

调试过程遇到的问题与解决

1. 男女分数统计下标错误：统计循环中判断 j（项目）而非 i（学校），修正后男女分数计算正确；
2. 冒泡排序交换逻辑缺失：比较总分/团体分后完整交换结构体，排序结果降序正确；
3. 查询项目时遍历范围错误：遍历全部学校而非项目，筛选得分大于 0 的学校输出；
4. 初始化遗漏：init 函数清空分数数组、重置各校总分男女分，避免多次运行数据残留；
5. 输入读取错乱：每个项目先读取排名类型 3/5，再依次录入获奖学校编号，匹配积分规则累加。

1.6.6 5. 小结

本实验采用结构体数组搭配二维顺序矩阵实现运动会分数统计系统，完成成绩录入、总分统计、多维度排序、双向查询全套功能。通过线性存储结构组织多类关联数据，掌握冒泡排序、多维数组遍历、结构体交换等基础操作，理解结构化数据在小型信息管理系统中的落地思路，熟练运用顺序存储完成批量数据统计与检索。

2 模块二

2.1 1. 二叉树

2.1.1 任务

建立二叉树，并实现二叉树的先序、中序、后序、层序遍历，求二叉树的宽度，统计二叉树中各类结点的个数，求根结点到指定结点的路径。功能要求：（1）建立二叉树；（2）对二叉树进行先序（递归、非递归）、中序（递归、非递归）、后序、层序遍历，并输出对应遍历序列（遍历可基于栈或队列来完成）；（3）求二叉树的高度、宽度；（4）统计二叉树中各类结点的个数；（5）求根结点到指定结点的路径。界面要求：程序运行后，给出菜单项的内容和输入提示：1. 建立二叉树 2. 遍历二叉树 3. 二叉树的高度、宽度 4. 各类结点的个数 5. 根结点到指定结点的路径 0. 退出请选择 0-5：

2.1.2 1. 需求分析

分析总结逻辑结构 二叉树由根结点、左子树和右子树递归构成，具有层次关系和递归特性。程序通过建立二叉链表结构存储二叉树，实现对结点及其逻辑关系的组织与管理。

分析总结运算集合 系统需完成二叉树建立、先序、中序、后序及层序遍历，高度与宽度计算，各类结点统计，以及根结点到指定结点路径查找等基本运算功能。

2.1.3 2. 概要设计

存储结构设计 采用二叉链表作为存储结构，每个结点包含数据域、左孩子指针和右孩子指针。遍历过程中利用顺序栈和顺序队列辅助实现非递归遍历与层序遍历功能。

算法设计思路 递归遍历依靠二叉树递归特性实现；非递归先、中序遍历借助栈模拟递归流程；层序遍历使用队列按层次依次访问；高度递归取左右子树最大值；宽度通过层序遍历记录每层最大结点数；结点统计递归区分叶子、度 1、度 2 结点；路径查找递归向下搜索，记录途经结点序列。

2.1.4 3. 详细设计

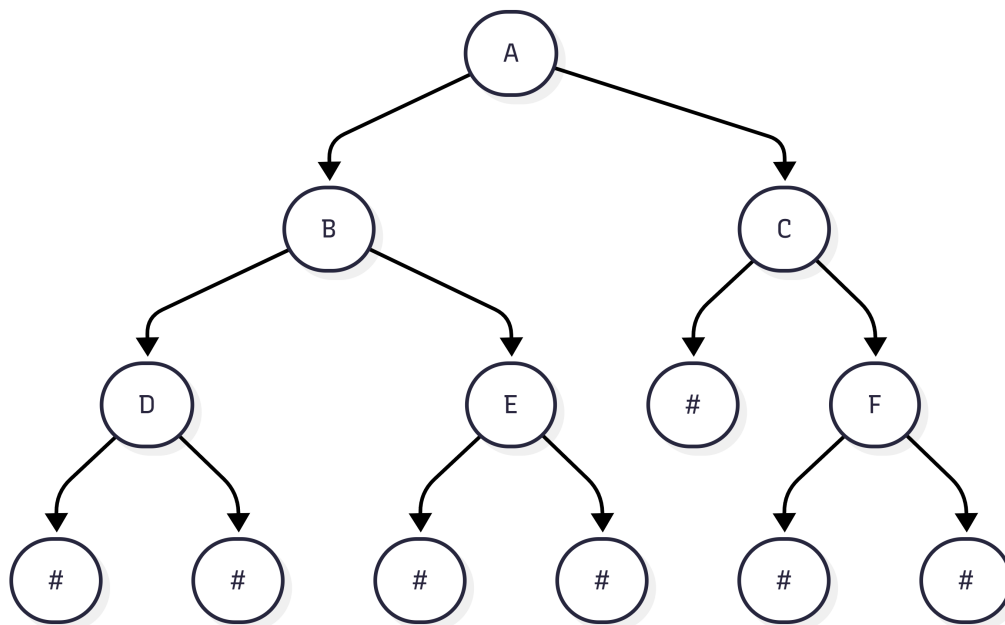


图 6: 二叉树

源代码（带注释） 代码：

```
#include <stdio.h>
#include <stdlib.h>

#define MAXSIZE 100

typedef struct node
{
    char data;
    struct node *lchild;
    struct node *rchild;
} Node, *BiTree;

//栈结构体
typedef struct
{
    BiTree data[MAXSIZE];
    int top;
} Stack;

//队列结构体
typedef struct
{
    BiTree data[MAXSIZE];
    int front;
```

```
    int rear;
} Queue;

//栈初始化
void InitStack(Stack *s)
{
    s->top = -1;
}

//判栈空
int StackEmpty(Stack *s)
{
    return s->top == -1;
}

//入栈
void Push(Stack *s, BiTree x)
{
    s->data[++s->top] = x;
}

//出栈
BiTree Pop(Stack *s)
{
    return s->data[s->top--];
}

//队列初始化
void InitQueue(Queue *q)
{
    q->front = q->rear = 0;
}

//判队空
int QueueEmpty(Queue *q)
{
    return q->front == q->rear;
}

//入队
void EnQueue(Queue *q, BiTree x)
{
    q->data[q->rear++] = x;
}
```

```

//出队
BiTree DeQueue(Queue *q)
{
    return q->data[q->front++];
}

//建立二叉树
BiTree CreateTree()
{
    char ch;
    scanf(" %c", &ch);

    if (ch == '#')
        return NULL;

    BiTree T = (BiTree)malloc(sizeof(Node));
    T->data = ch;

    T->lchild = CreateTree();
    T->rchild = CreateTree();

    return T;
}

//菜单打印
void Menu()
{
    printf("\n=====二叉树操作菜单=====\n");
    printf("1. 建立二叉树\n");
    printf("2. 遍历二叉树\n");
    printf("3. 二叉树的高度、宽度\n");
    printf("4. 各类结点的个数\n");
    printf("5. 根结点到指定结点的路径\n");
    printf("0. 退出\n");
    printf("请选择0-5: ");
}

//递归先序遍历
void PreOrder(BiTree T)
{
    if (T)
    {
        printf("%c ", T->data);
    }
}

```

```
        PreOrder(T->lchild);
        PreOrder(T->rchild);
    }
}

//递归中序遍历
void InOrder(BiTree T)
{
    if (T)
    {
        InOrder(T->lchild);
        printf("%c ", T->data);
        InOrder(T->rchild);
    }
}

//递归后序遍历
void PostOrder(BiTree T)
{
    if (T)
    {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        printf("%c ", T->data);
    }
}

//非递归先序遍历
void PreOrder_NR(BiTree T)
{
    Stack stack;
    InitStack(&stack);

    while (T || !StackEmpty(&stack))
    {
        while (T)
        {
            printf("%c ", T->data);
            Push(&stack, T);
            T = T->lchild;
        }

        if (!StackEmpty(&stack))
        {
```

```

        T = Pop(&stack);
        T = T->rchild;
    }
}

//非递归中序遍历
void InOrder_NR(BiTree T)
{
    Stack stack;
    InitStack(&stack);

    while (T || !StackEmpty(&stack))
    {
        while (T)
        {
            Push(&stack, T);
            T = T->lchild;
        }

        if (!StackEmpty(&stack))
        {
            T = Pop(&stack);
            printf("%c ", T->data);
            T = T->rchild;
        }
    }
}

```

```

//层序遍历
void LevelOrder(BiTree T)
{
    if (!T)
        return;
    Queue queue;
    InitQueue(&queue);

    EnQueue(&queue, T);

    while (!QueueEmpty(&queue))
    {
        BiTree p = DeQueue(&queue);

        printf("%c ", p->data);
    }
}

```

```

        if (p->lchild)
            EnQueue(&queue, p->lchild);

        if (p->rchild)
            EnQueue(&queue, p->rchild);
    }
}

//求高度
int Height(BiTree T)
{
    if (T == NULL)
        return 0;
    int lh = Height(T->lchild);
    int rh = Height(T->rchild);

    return (lh > rh ? lh : rh) + 1;
}

//求宽度
int Width(BiTree T)
{
    if (!T)
        return 0;
    Queue queue;
    InitQueue(&queue);

    int maxWidth = 0;
    EnQueue(&queue, T);

    while (!QueueEmpty(&queue))
    {
        int count = queue.rear - queue.front;

        if (count > maxWidth)
            maxWidth = count;
        for (int i = 0; i < count; i++)
        {
            BiTree p = DeQueue(&queue);

            if (p->lchild)
                EnQueue(&queue, p->lchild);

```

```

        if (p->rchild)
            EnQueue(&queue, p->rchild);
    }
}

return maxWidth;
}

//统计结点
void CountNodes(BiTree T, int *total, int *leaf, int *degree1, int *degree2)
{
    if (!T)
        return;

    (*total)++;

    int left = (T->lchild != NULL);
    int right = (T->rchild != NULL);

    if (!left && !right)
        (*leaf)++;
    else if (left + right == 1)
        (*degree1)++;
    else
        (*degree2)++;

    CountNodes(T->lchild, total, leaf, degree1, degree2);
    CountNodes(T->rchild, total, leaf, degree1, degree2);
}

//求路径
int FindPath(BiTree T, char target, char path[], int depth)
{
    if (!T)
        return 0;

    path[depth] = T->data;

    if (T->data == target)
    {
        printf("路径: ");
        for (int i = 0; i <= depth; i++)
            printf("%c ", path[i]);
        printf("\n");
    }
}

```

```

        return 1;
    }

    if (FindPath(T->lchild, target, path, depth + 1))
        return 1;
    if (FindPath(T->rchild, target, path, depth + 1))
        return 1;

    return 0;
}

//释放树
void DestroyTree(BiTree T)
{
    if (T)
    {
        DestroyTree(T->lchild);
        DestroyTree(T->rchild);
        free(T);
    }
}

int main()
{
    BiTree T = NULL;
    int choice;

    while (1)
    {
        Menu();
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
            {
                if (T)
                {
                    DestroyTree(T);
                    T = NULL;
                }
                printf("请输入先序序列(#表示空结点):\n");
                T = CreateTree();
                printf("二叉树建立成功! \n");
            }
        }
    }
}

```

```

        break;
    }
    case 2:
    {
        if (!T)
        {
            printf("请先建立二叉树! \n");
            break;
        }
        printf("\n递归先序遍历:\n");
        PreOrder(T);
        printf("\n\n非递归先序遍历:\n");
        PreOrder_NR(T);
        printf("\n\n递归中序遍历:\n");
        InOrder(T);
        printf("\n\n非递归中序遍历:\n");
        InOrder_NR(T);
        printf("\n\n后序遍历:\n");
        PostOrder(T);
        printf("\n\n层序遍历:\n");
        LevelOrder(T);
        printf("\n");
        break;
    }
    case 3:
    {
        if (!T)
        {
            printf("请先建立二叉树! \n");
            break;
        }
        printf("二叉树高度=%d\n", Height(T));
        printf("二叉树宽度=%d\n", Width(T));
        break;
    }
    case 4:
    {
        if (!T)
        {
            printf("请先建立二叉树! \n");
            break;
        }
        int total = 0, leaf = 0, degree1 = 0, degree2 = 0;
        CountNodes(T, &total, &leaf, &degree1, &degree2);
    }
}

```

```

        printf("总结点数=%d\n", total);
        printf("叶子结点=%d\n", leaf);
        printf("度为1结点=%d\n", degree1);
        printf("度为2结点=%d\n", degree2);
        break;
    }
    case 5:
    {
        if (!T)
        {
            printf("请先建立二叉树! \n");
            break;
        }
        char target;
        printf("请输入目标结点:");
        scanf(" %c", &target);
        char path[MAXSIZE];
        if (!FindPath(T, target, path, 0))
            printf("未找到该结点!\n");
        break;
    }
    case 0:
    {
        DestroyTree(T);
        printf("程序结束! \n");
        return 0;
    }
    default:
        printf("输入错误, 请重新输入! \n");
    }
}
return 0;
}

```

2.1.5 4. 调试分析

测试输入与完整运行输出

=====二叉树操作菜单=====

1. 建立二叉树
 2. 遍历二叉树
 3. 二叉树的高度、宽度
 4. 各类结点的个数
 5. 根结点到指定结点的路径
 0. 退出
- 请选择0-5: 1

请输入先序序列(#表示空结点):

ABD##E##C#F##

二叉树建立成功!

=====二叉树操作菜单=====

1. 建立二叉树
2. 遍历二叉树
3. 二叉树的高度、宽度
4. 各类结点的个数
5. 根结点到指定结点的路径
0. 退出

请选择0-5: 2

递归先序遍历:

A B D E C F

非递归先序遍历:

A B D E C F

递归中序遍历:

D B E A C F

非递归中序遍历:

D B E A C F

后序遍历:

D E B F C A

层序遍历:

A B C D E F

=====二叉树操作菜单=====

1. 建立二叉树
2. 遍历二叉树
3. 二叉树的高度、宽度
4. 各类结点的个数
5. 根结点到指定结点的路径
0. 退出

请选择0-5: 3

二叉树高度=3

二叉树宽度=3

=====二叉树操作菜单=====

1. 建立二叉树

```
2. 遍历二叉树
3. 二叉树的高度、宽度
4. 各类结点的个数
5. 根结点到指定结点的路径
0. 退出
请选择0-5: 4
总结点数=6
叶子结点=3
度为1结点=1
度为2结点=2
```

```
=====二叉树操作菜单=====
1. 建立二叉树
2. 遍历二叉树
3. 二叉树的高度、宽度
4. 各类结点的个数
5. 根结点到指定结点的路径
0. 退出
请选择0-5: 5
请输入目标结点:F
路径: A C F
```

```
=====二叉树操作菜单=====
1. 建立二叉树
2. 遍历二叉树
3. 二叉树的高度、宽度
4. 各类结点的个数
5. 根结点到指定结点的路径
0. 退出
请选择0-5: 0
程序结束!
```

调试过程遇到的问题与解决

1. 缺少 Menu 菜单函数，main 调用报错：补充 Menu 函数打印操作菜单，程序菜单界面正常显示；
2. 多次新建树内存泄漏：新建树前调用 DestroyTree 释放原有二叉树，避免堆内存堆积；
3. 路径查找数组越界：设置 MAXSIZE 足够大的路径数组，递归传递 depth 记录当前层数；
4. 层序宽度统计每层数量错误：每次循环先记录当前队列元素总数，完整遍历一层再入队孩子；
5. 结点统计度分类逻辑出错：区分左右孩子存在情况，正确划分叶子、度 1、度 2 三类结点；

2.1.6 5. 小结

本实验采用二叉链表链式存储结构构建二叉树，实现递归与非递归先、中序遍历，后序、层序遍历，完成树高度、宽度求解，叶子/度 1/度 2 结点数量统计，以及根到目标结点路径检索全套功能。借助栈、

队列辅助实现非递归遍历与层序遍历，充分运用递归思想处理二叉树天然分层递归结构，加深对二叉树存储、遍历、递归回溯算法的理解，熟练掌握链式二叉树基础操作与线性辅助容器（栈、队列）的综合运用。

2.2 2. 哈夫曼编码

2.2.1 任务

要求对输入的一串文本字符实现哈夫曼编码，再对哈夫曼编码生成的 0、1 串进行译码，输出原始文本字符串。功能要求：（1）统计文本字符串中不同字符的个数以及各字符在文本中出现的次数；（2）根据字符数量与对应频度建立并打印哈夫曼树；（3）生成并输出每个字符对应的哈夫曼前缀编码；（4）将原始文本转换为连续 01 二进制编码序列；（5）输入 01 编码串，基于哈夫曼树译码还原原始文本。界面要求：程序运行后，输出功能菜单并提示输入选项：统计字符的个数以及字符出现的次数建立并输出哈夫曼树设计并输出哈夫曼编码将文本转换成 01 编码将 01 编码翻译成文本退出请选择 0-5：

2.2.2 1. 需求分析

分析总结逻辑结构 字符频度使用一维线性数组存储；哈夫曼树采用静态结构体数组存储，属于顺序存储二叉树；各字符哈夫曼编码存放在二维字符数组；原始文本、二进制编码串使用一维字符数组线性存储。

分析总结运算集合 字符频度统计、选取权值最小两个结点、哈夫曼树构造、叶子回溯生成前缀编码、明文批量编码、二进制编码逐位译码、菜单交互控制。

2.2.3 2. 概要设计

存储结构设计 静态哈夫曼结点结构体 HTNode：包含字符、权值、双亲下标、左孩子下标、右孩子下标；一维数组 chars 保存去重字符，freq 数组保存对应字符出现次数；二维字符数组 HC 存储每个叶子字符的 01 哈夫曼编码。

算法设计思路 频度统计：遍历输入字符串，查重累加每个字符出现次数；构造哈夫曼树：循环选出两个无父结点的最小权值结点，合并生成新中间结点，更新父子关系；生成编码：从叶子结点向上回溯至根结点，左分支记'0'、右分支记'1'，逆序拼接得到标准编码；文本编码：遍历原文每个字符，匹配对应编码拼接成完整 01 串；译码：从根结点逐位读取 0/1 向下遍历树，到达叶子则输出字符并重置指针回到根，继续解析下一组编码。

2.2.4 3. 详细设计

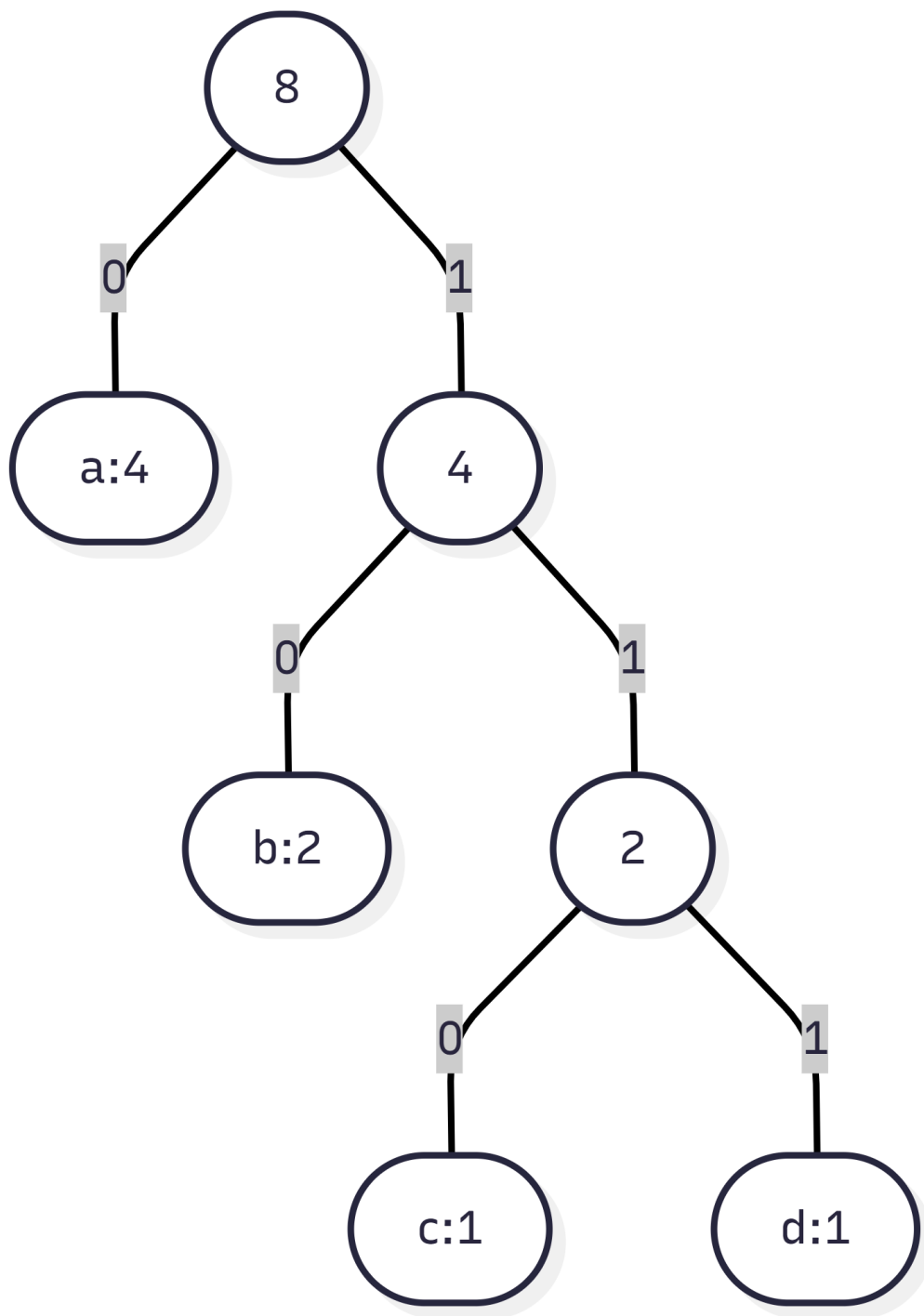


图 7: 哈夫曼编码示意图

源代码（带注释） 代码：

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

#define MAXSIZE 100
```

```
typedef struct
{
    char ch;          // 字符
    int weight;       // 权值(频率)
    int parent;
    int lchild;
    int rchild;

} HTNode;

// 统计字符频率
void CountChar(char str[], char chars[], int freq[], int *count)
{
    *count = 0;

    for (int i = 0; str[i]; i++)
    {
        int found = -1;

        for (int j = 0; j < *count; j++)
        {
            if (chars[j] == str[i])
            {
                found = j;
                break;
            }
        }

        if (found == -1)
        {
            chars[*count] = str[i];
            freq[*count] = 1;
            (*count)++;
        }
        else
        {
            freq[found]++;
        }
    }
}

// 选两个最小权值结点
void Select(HTNode ht[], int end, int *s1, int *s2)
{
```

```

    *s1 = *s2 = 0;

    for (int i = 1; i <= end; i++)
    {
        if (ht[i].parent != 0)
            continue;

        if (*s1 == 0 || ht[i].weight < ht[*s1].weight)
            *s1 = i;
    }

    for (int i = 1; i <= end; i++)
    {
        if (ht[i].parent != 0 || i == *s1)
            continue;

        if (*s2 == 0 || ht[i].weight < ht[*s2].weight)
            *s2 = i;
    }

    if (ht[*s1].weight > ht[*s2].weight)
    {
        int t = *s1;
        *s1 = *s2;
        *s2 = t;
    }
}

// 建立哈夫曼树
void CreateHuffmanTree(HTNode ht[], char chars[], int freq[], int n)
{
    int m = 2 * n - 1;

    for (int i = 1; i <= m; i++)
    {
        ht[i].parent = 0;
        ht[i].lchild = 0;
        ht[i].rchild = 0;
        ht[i].weight = 0;
        ht[i].ch = '\\0';
    }

    for (int i = 1; i <= n; i++)
    {

```

```

        ht[i].ch = chars[i - 1];
        ht[i].weight = freq[i - 1];
    }

    for (int i = n + 1; i <= m; i++)
    {
        int s1, s2;

        Select(ht, i - 1, &s1, &s2);

        ht[s1].parent = i;
        ht[s2].parent = i;

        ht[i].lchild = s1;
        ht[i].rchild = s2;

        ht[i].weight = ht[s1].weight + ht[s2].weight;
    }
}

// 输出哈夫曼树
void PrintTree(HTNode ht[], int m)
{
    printf("\n哈夫曼树: \n\n");

    printf("编号\t字符\t权值\t双亲\t左孩子\t右孩子\n");

    for (int i = 1; i <= m; i++)
    {
        printf("%d\t", i);

        if (ht[i].ch)
            printf("%c\t", ht[i].ch);
        else
            printf("-\t");

        printf("%d\t%d\t%d\t%d\n", ht[i].weight, ht[i].parent, ht[i].lchild, ht[i].rchild);
    }
}

// 生成哈夫曼编码
void CreateCode(HTNode ht[], char HC[][MAXSIZE], int n)
{
    char code[MAXSIZE];

```

```

    for (int i = 1; i <= n; i++)
    {
        int start = MAXSIZE - 1;

        code[start] = '\0';

        int c = i;
        int p = ht[c].parent;

        while (p)
        {
            if (ht[p].lchild == c)
                code[--start] = '0';
            else
                code[--start] = '1';

            c = p;
            p = ht[c].parent;
        }

        strcpy(HC[i], &code[start]);
    }
}

// 输出编码
void PrintCode(char chars[], char HC[][MAXSIZE], int n)
{
    printf("\n哈夫曼编码: \n\n");

    for (int i = 1; i <= n; i++)
    {
        printf("%c : %s\n", chars[i - 1], HC[i]);
    }
}

// 编码
void Encode(char str[], char chars[], char HC[][MAXSIZE], int n, char result[])
{
    result[0] = '\0';

    for (int i = 0; str[i]; i++)
    {
        for (int j = 1; j <= n; j++)

```

```

        {
            if (str[i] == chars[j - 1])
            {
                strcat(result, HC[j]);
                break;
            }
        }
    }
}

// 译码
void Decode(HTNode ht[], int root, char code[])
{
    printf("\n译码结果: \n");

    int p = root;

    for (int i = 0; code[i]; i++)
    {
        if (code[i] == '0')
            p = ht[p].lchild;
        else
            p = ht[p].rchild;

        if (ht[p].lchild == 0 &&
            ht[p].rchild == 0)
        {
            printf("%c", ht[p].ch);
            p = root;
        }
    }

    printf("\n");
}

void Menu()
{
    // 打印菜单
    printf("1. 统计字符的个数以及字符出现的次数\n");
    printf("2. 建立并输出哈夫曼树\n");
    printf("3. 设计并输出哈夫曼编码\n");
    printf("4. 将文本转换成01编码\n");
    printf("5. 将01编码翻译成文本\n");
    printf("0. 退出\n");
}

```

```
    printf("请选择0-5: ");
}
int main()
{
    int choice = 0;
    char str[MAXSIZE];
    char chars[MAXSIZE];
    int freq[MAXSIZE];
    int count;
    HTNode ht[2 * MAXSIZE];
    char HC[MAXSIZE][MAXSIZE];
    char codeResult[1000];
    // 标记是否已经初始化数据
    int hasData = 0;
    while (1)
    {
        Menu();
        scanf("%d", &choice);

        if (choice == 0)
        {
            printf("程序退出! \n");
            break;
        }

        if (choice == 1)
        {
            // 1、输入字符串 + 统计频率
            printf("请输入字符串: ");
            scanf("%s", str);
            CountChar(str, chars, freq, &count);
            hasData = 1;

            printf("\n字符频率统计: \n\n");
            for (int i = 0; i < count; i++)
            {
                printf("%c : %d\n", chars[i], freq[i]);
            }
        }
        else
        {
            // 2~5必须先执行1录入数据
            if (!hasData)
            {
```

```

        printf("错误：请先选择1录入字符串并统计字符！\n");
        continue;
    }
    if (choice == 2)
    {
        // 2、建树并输出树表
        CreateHuffmanTree(ht, chars, freq, count);
        PrintTree(ht, 2 * count - 1);
    }
    else if (choice == 3)
    {
        // 3、生成并输出编码
        CreateHuffmanTree(ht, chars, freq, count);
        CreateCode(ht, HC, count);
        PrintCode(chars, HC, count);
    }
    else if (choice == 4)
    {
        // 4、原文转为01串
        CreateHuffmanTree(ht, chars, freq, count);
        CreateCode(ht, HC, count);
        Encode(str, chars, HC, count, codeResult);
        printf("\n编码结果：\n%s\n", codeResult);
    }
    else if (choice == 5)
    {
        // 5、01串译码
        CreateHuffmanTree(ht, chars, freq, count);
        CreateCode(ht, HC, count);
        Encode(str, chars, HC, count, codeResult);
        Decode(ht, 2 * count - 1, codeResult);
    }
    else
    {
        printf("输入选项无效，请输入0~5！\n");
    }
}

return 0;
}

```

2.2.5 4. 调试分析

测试输入与完整运行输出

统计字符的个数以及字符出现的次数

建立并输出哈夫曼树

设计并输出哈夫曼编码

将文本转换成 01 编码

将 01 编码翻译成文本

退出

请选择 0-5: 1

请输入字符串: abacabda

字符频率统计:

a : 4

b : 2

c : 1

d : 1

统计字符的个数以及字符出现的次数

建立并输出哈夫曼树

设计并输出哈夫曼编码

将文本转换成 01 编码

将 01 编码翻译成文本

退出

请选择 0-5: 2

哈夫曼树:

编号 字符 权值 双亲 左孩子 右孩子

1 a 4 7 0 0

2 b 2 6 0 0

3 c 1 5 0 0

4 d 1 5 0 0

5 - 2 6 3 4

6 - 4 7 2 5

7 - 8 0 1 6

统计字符的个数以及字符出现的次数

建立并输出哈夫曼树

设计并输出哈夫曼编码

将文本转换成 01 编码

将 01 编码翻译成文本

退出

请选择 0-5: 3

哈夫曼编码:

a : 0

b : 10

c : 110

d : 111

统计字符的个数以及字符出现的次数

建立并输出哈夫曼树

设计并输出哈夫曼编码

将文本转换成 01 编码
将 01 编码翻译成文本
退出
请选择 0-5: 4
编码结果:
01001100101110
统计字符的个数以及字符出现的次数
建立并输出哈夫曼树
设计并输出哈夫曼编码
将文本转换成 01 编码
将 01 编码翻译成文本
退出
请选择 0-5: 5
译码结果:
abacabda
统计字符的个数以及字符出现的次数
建立并输出哈夫曼树
设计并输出哈夫曼编码
将文本转换成 01 编码
将 01 编码翻译成文本
退出
请选择 0-5: 0
程序退出!

调试过程遇到的问题与解决

1. 未统计字符直接执行编码 / 译码功能: 增加全局标记变量 `hasData` 拦截非法操作, 提示先执行字符统计功能;
2. `Select` 函数选出的两个最小结点顺序颠倒: 比较权值后交换 `s1`、`s2`, 保证左子树权值更小;
3. 生成哈夫曼编码字符顺序颠倒: 从叶子向上遍历至根结点, 倒序存入临时数组, 截取有效起始位置复制得到正确编码;
4. 译码识别单个字符后无法连续解析下一个字符: 每次遍历到叶子结点输出字符后, 将遍历指针重置回根结点;
5. 多次重复执行建树功能数据覆盖: 重复调用 `CreateHuffmanTree` 会完整重置树数组, 无逻辑错误, 不影响运行结果;
6. 仅支持无空格文本输入

2.2.6 5. 小结

本实验实现了哈夫曼编码完整流程, 掌握贪心算法构造最优二叉树、前缀编码生成及译码方法, 理解了数据压缩基本原理, 提升了树结构应用能力。

2.3 3. 并查集

2.3.1 任务

用树形结构来表示集合，树的每个结点代表一个集合元素。实现查找（判定）元素所属集合的运算、实现集合的并运算。功能要求：（1）建立并输出集合的存储结构；（2）设计并实现查找（判定）元素所属集合的运算；（3）设计并实现集合的并运算。界面要求：程序运行后，给出菜单项的内容和输入提示：1. 2. 3. 0. 建立集合查找（判定）元素所属集合集合的并运算退出请选择 0-3:

2.3.2 1. 需求分析

分析总结逻辑结构 采用树形逻辑结构存储集合，每个节点代表集合元素，根节点为集合代表元。

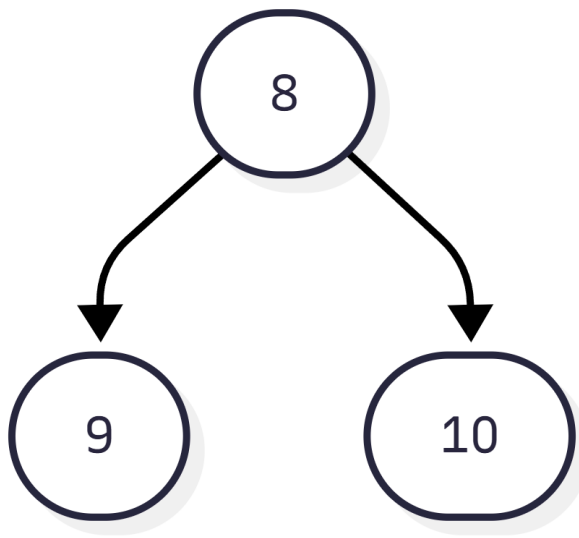
分析总结运算集合 实现集合初始化构建、元素根节点查找、两个集合合并三大基础运算，搭配菜单交互使用。

2.3.3 2. 概要设计

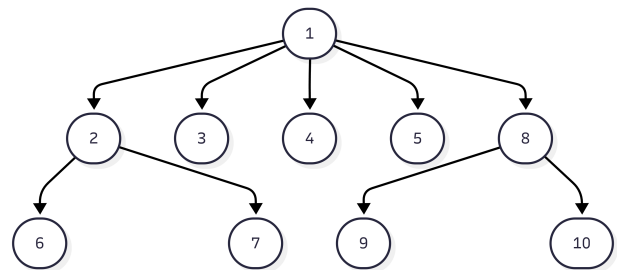
存储结构设计 采用一维数组存储父节点，下标为元素，数组值为父节点，实现树形静态存储。find 函数带路径压缩优化查找，unite 函数按根合并集合，模块化设计各功能子程序。

算法设计思路

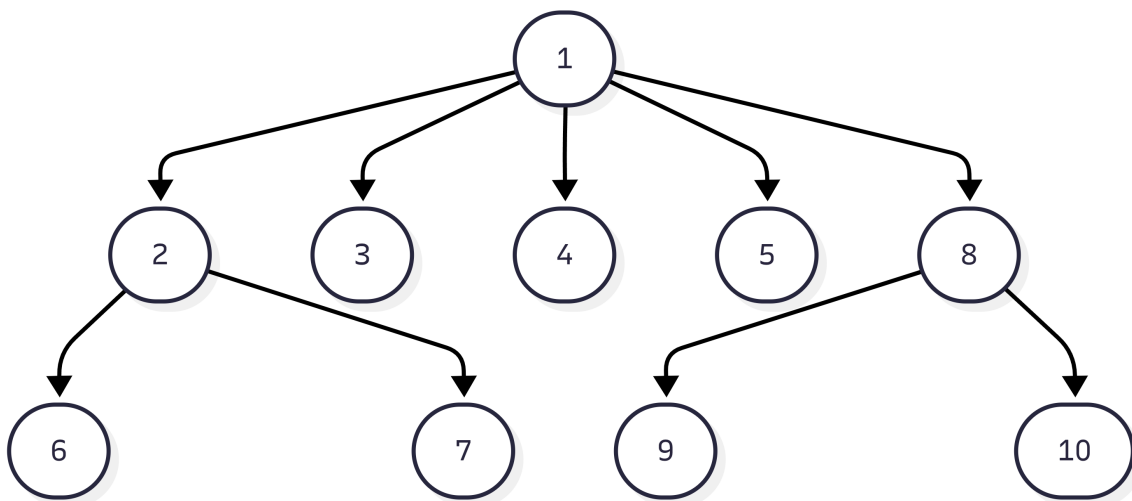
2.3.4 3. 详细设计



(a) 并查集 1



(b) 并查集 2



(c) 合并后的并查集

图 8: 并查集示意图

源代码（带注释） 代码：

```
#include <stdio.h>
#define N 10010
int father[N];
// 初始化集合
void init(int n)
{
    for (int i = 1; i <= n; i++)
        father[i] = i;
}
```

// 查找（路径压缩）

```
int find(int x)
{
    if (father[x] == x)
        return x;
    return father[x] = find(father[x]);
}

// 合并集合
void unite(int x, int y)
{
    int fx = find(x);
    int fy = find(y);

    if (fx != fy)
        father[fx] = fy;
}

// 输出当前集合结构（按根分类简单展示）
void printSets(int n)
{
    printf("\n当前集合结构: \n");

    for (int i = 1; i <= n; i++)
        printf("%d -> %d\n", i, find(i));
}

// 查询元素所属集合
void query(int x)
{
    printf("元素 %d 所属集合代表元: %d\n", x, find(x));
}

int main()
{
    int n = 0;
    int choice;

    while (1)
    {
        printf("\n===== 并查集菜单 =====\n");
        printf("1. 建立集合\n");
        printf("2. 查找（判定）元素所属集合\n");
        printf("3. 集合的并运算\n");
        printf("4. 输出集合结构\n");
```

```
printf("0. 退出\n");
printf("请选择 0-4: ");
scanf("%d", &choice);

if (choice == 0)
    break;

if (choice == 1)
{
    printf("输入元素个数 n: ");
    scanf("%d", &n);
    init(n);
    printf("集合建立完成! \n");
}
else if (choice == 2)
{
    int x;
    printf("输入元素 x: ");
    scanf("%d", &x);
    query(x);
}
else if (choice == 3)
{
    int x, y;
    printf("输入要合并的两个元素 x y: ");
    scanf("%d %d", &x, &y);
    unite(x, y);
    printf("合并完成! \n");
}
else if (choice == 4)
{
    printSets(n);
}
else
{
    printf("无效选择! \n");
}

return 0;
}
```

2.3.5 4. 调试分析

测试输入与完整运行输出

===== 并查集菜单 =====

1. 建立集合
2. 查找（判定）元素所属集合
3. 集合的并运算
4. 输出集合结构
0. 退出

输入元素个数 **n**: 5

集合建立完成！

===== 并查集菜单 =====

1. 建立集合
2. 查找（判定）元素所属集合
3. 集合的并运算
4. 输出集合结构
0. 退出

请选择 0-4: 3

输入要合并的两个元素 **x y**: 1 2

合并完成！

===== 并查集菜单 =====

1. 建立集合
2. 查找（判定）元素所属集合
3. 集合的并运算
4. 输出集合结构
0. 退出

请选择 0-4: 3

输入要合并的两个元素 **x y**: 2 3

合并完成！

2.3.6 5. 小结

本实验实现并查集的查找、合并与路径压缩操作，可高效处理元素连通性判定，理解集合分治思想，优化查询效率。

3 模块三

3.1 1. 最小代价生成树

3.1.1 任务

建立图并求图的最小代价生成树。功能要求：（1）建立图的存储结构（邻接矩阵或邻接表），能够输入图的顶点、边，以及边上的权值的信息，存储到相应存储结构中，并输出图的相关信息。（2）利用 Prim 算法或 Kruskal 算法求图的最小代价生成树。

3.1.2 1. 需求分析

分析总结逻辑结构 逻辑结构采用邻接矩阵存储无向带权图，顶点与边为二维线性结构。

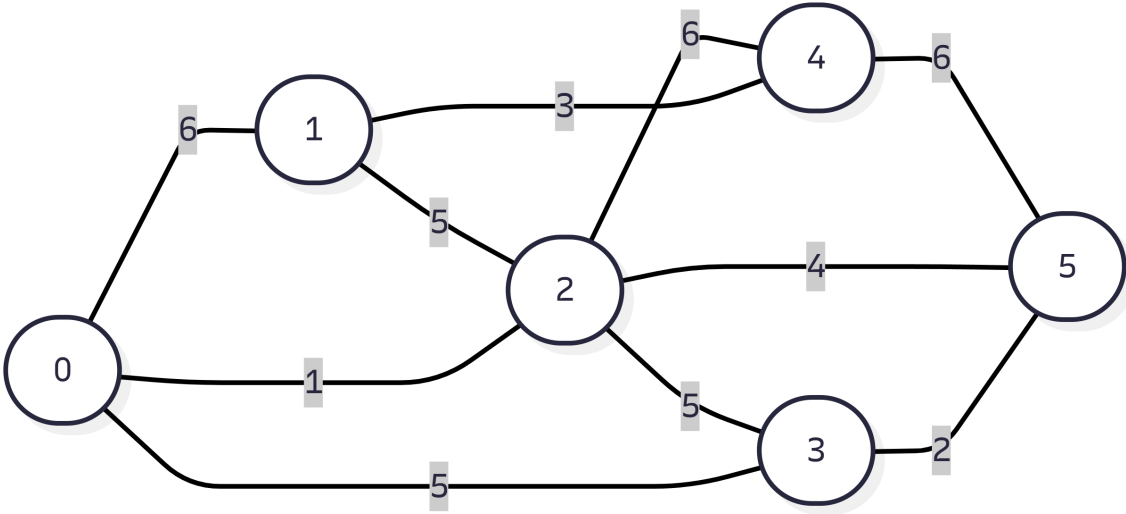
分析总结运算集合 运算包含图构建、矩阵输出、Prim 与 Kruskal 生成最小生成树，辅以边排序、并查集判连通。

3.1.3 2. 概要设计

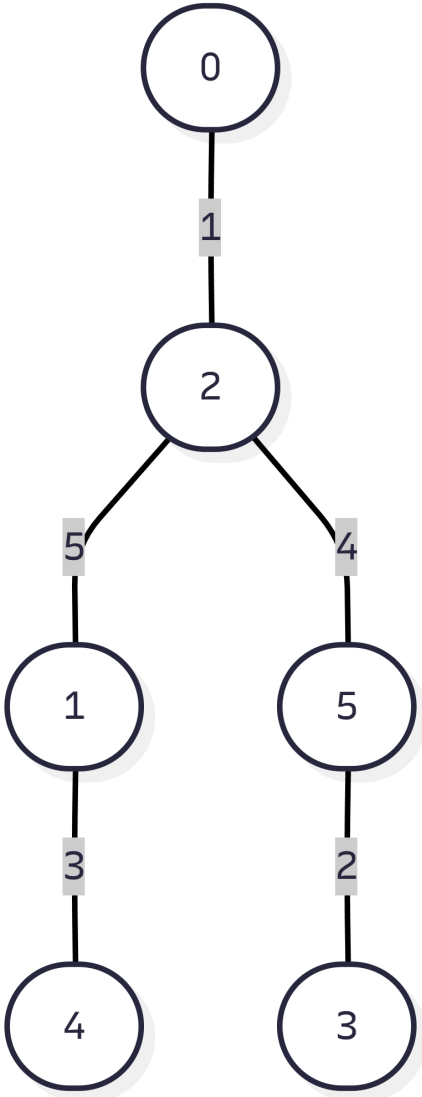
存储结构设计 存储用邻接矩阵存图，Edge 结构体存边，数组实现并查集。

算法设计思路 Prim 借助 lowcost 数组选最短邻边；Kruskal 快排边，配合并查集规避环路生成生成树。

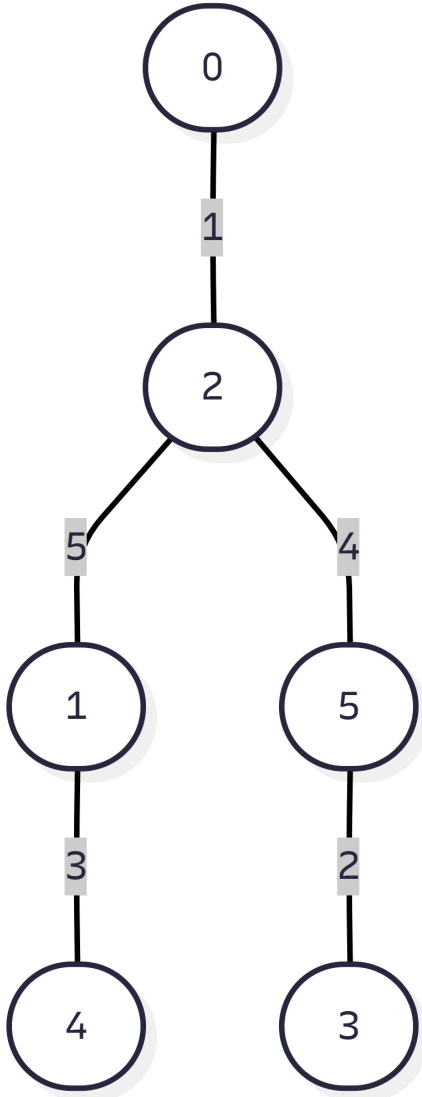
3.1.4 3. 详细设计



(a) 图



(b) prime



(c) kruskal

图 9: 最小生成树

源代码（带注释） 代码：

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define MAXV 100
#define INF 32767

// 图结构

//邻接矩阵(prim算法)
typedef struct
{
    int vexnum;
    int arcnum;
    int arcs[MAXV][MAXV];
} MGraph;

// 边结构(Kruskal算法)

typedef struct
{
    int begin;
    int end;
    int weight;
} Edge;

// 并查集

int parent[MAXV];

int Find(int x)
{
    if(parent[x] == x)
        return parent[x];
    else
        return parent[x] = Find(x);
}

void Union(int x, int y)
{
    int fx = Find(x);
    int fy = Find(y);

    if (fx != fy)
```

```
        parent[fx] = fy;
    }

// 建图

void CreateGraph(MGraph *G)
{
    int i, j;
    int u, v, w;

    printf("输入顶点数和边数: ");
    scanf("%d%d", &G->vexnum, &G->arcnum);

    for (i = 0; i < G->vexnum; i++)
    {
        for (j = 0; j < G->vexnum; j++)
        {
            if (i == j)
                G->arcs[i][j] = 0;
            else
                G->arcs[i][j] = INF;
        }
    }

    printf("输入边(起点 终点 权值): \n");

    for (i = 0; i < G->arcnum; i++)
    {
        scanf("%d%d%d", &u, &v, &w);

        G->arcs[u][v] = w;
        G->arcs[v][u] = w;
    }
}

// 输出邻接矩阵

void PrintGraph(MGraph G)
{
    int i, j;

    printf("\n邻接矩阵: \n");

    for (i = 0; i < G.vexnum; i++)
```

```

{
    for (j = 0; j < G.vexnum; j++)
    {
        //其中的N表示的是无穷大
        if (G.arcs[i][j] == INF)
            printf("N ");
        else
            printf("%d ", G.arcs[i][j]);
    }
    printf("\n");
}

}

// Prim算法

void Prim(MGraph G, int start)
{
    int lowcost[MAXV];
    int closest[MAXV];

    int i, j;
    int min;
    int k;

    int sum = 0;

    for (i = 0; i < G.vexnum; i++)
    {
        lowcost[i] = G.arcs[start][i];
        closest[i] = start;
    }

    lowcost[start] = 0;

    printf("\nPrim最小生成树: \n");

    for (i = 1; i < G.vexnum; i++)
    {
        min = INF;
        k = -1;

        for (j = 0; j < G.vexnum; j++)
        {
            if (lowcost[j] != 0 && lowcost[j] < min)

```

```

        {
            min = lowcost[j];
            k = j;
        }
    }

    printf("%d -- %d  权值=%d\n", closest[k], k, min);

    sum += min;

    lowcost[k] = 0;

    for (j = 0; j < G.vexnum; j++)
    {
        if (lowcost[j] != 0 && G.arcs[k][j] < lowcost[j])
        {
            lowcost[j] = G.arcs[k][j];
            closest[j] = k;
        }
    }
}

printf("最小生成树总权值=%d\n", sum);
}

// 边排序

int get_random_index(int left, int right)
{
    return rand() % (right - left + 1) + left;
}

void swap(Edge *a, Edge *b)
{
    Edge temp = *a;
    *a = *b;
    *b = temp;
}

void SortEdges(Edge edges[], int left, int right)
{
    if (left >= right)
        return;

```

```

    int index = get_random_index(left, right);

    int key = edges[index].weight;

    int l = left - 1;
    int i = left;
    int r = right + 1;

    while (i < r)
    {
        if (edges[i].weight < key)
        {
            swap(&edges[++l], &edges[i++]);
        }
        else if (edges[i].weight == key)
        {
            i++;
        }
        else
        {
            swap(&edges[--r], &edges[i]);
        }
    }

    SortEdges(edges, left, l);
    SortEdges(edges, r, right);
}

// Kruskal算法

void Kruskal(MGraph G)
{
    Edge edges[MAXV * MAXV];

    int edgeCount = 0;

    int i, j;

    // 提取边
    for (i = 0; i < G.vexnum; i++)
    {
        for (j = i + 1; j < G.vexnum; j++)
        {
            if (G.arcs[i][j] != INF && G.arcs[i][j] != 0)

```

```

        {
            edges[edgeCount].begin = i;
            edges[edgeCount].end = j;
            edges[edgeCount].weight = G.arcs[i][j];

            edgeCount++;
        }
    }

SortEdges(edges, 0, edgeCount - 1);

for (i = 0; i < G.vexnum; i++)
    parent[i] = i;

int sum = 0;
int cnt = 0;

printf("\nKruskal最小生成树: \n");

for (i = 0; i < edgeCount; i++)
{
    int u = edges[i].begin;
    int v = edges[i].end;
    int w = edges[i].weight;

    if (Find(u) != Find(v))
    {
        Union(u, v);

        printf("%d -- %d 权值=%d\n", u, v, w);

        sum += w;
        cnt++;

        if (cnt == G.vexnum - 1)
            break;
    }
}

printf("最小生成树总权值=%d\n", sum);
}

int main()

```

```
{
    srand(time(NULL));
    MGraph G;

    CreateGraph(&G);

    PrintGraph(G);

    Prim(G, 0);

    Kruskal(G);

    return 0;
}
```

3.1.5 4. 调试分析

测试输入与完整运行输出

输入顶点数和边数：6 10

输入边(起点 终点 权值)：

0 1 6

0 3 5

0 2 1

1 2 5

1 4 3

2 3 5

2 4 6

2 5 4

3 5 2

4 5 6

邻接矩阵：

0 6 1 5 N N

6 0 5 N 3 N

1 5 0 5 6 4

5 N 5 0 N 2

N 3 6 N 0 6

N N 4 2 6 0

Prim最小生成树：

0 -- 2 权值=1

2 -- 5 权值=4

5 -- 3 权值=2

2 -- 1 权值=5

1 -- 4 权值=3
最小生成树总权值=15

Kruskal最小生成树:

0 -- 2 权值=1
3 -- 5 权值=2
1 -- 4 权值=3
2 -- 5 权值=4

3.1.6 5. 小结

本实验实现两种最小生成树算法，掌握邻接矩阵建图，理解 Prim 贪心找点、Kruskal 贪心选边思路，熟练运用并查集解决环路判断问题。

3.2 2. 拓扑排序及关键路径

3.2.1 任务

建立图并实现图的拓扑排序和关键路径。功能要求：（1）建立图的存储结构（邻接矩阵或邻接表），能够输入图的顶点和边以及边上的权值等信息，存储到相应存储结构中，并输出图的结构。（2）对图进行拓扑排序并输出结果。（3）在（2）的基础上求出图的关键路径并输出关键路径以及源点到汇点的最长路径长度。

3.2.2 1. 需求分析

分析总结逻辑结构 本实验采用邻接表存储带权有向图，实现图结构建立与输出。

分析总结运算集合

3.2.3 2. 概要设计

主要运算包括拓扑排序判环、顶点发生时间计算、筛选关键活动并求解工程关键路径与总工期。

存储结构设计 采用邻接表存储有向图，记录顶点入度。利用栈实现拓扑排序，正向遍历求顶点最早时间，反向遍历求最迟时间，通过时间差值筛选关键路径。

算法设计思路

3.2.4 3. 详细设计

源代码（带注释） 代码：

```
#include<stdio.h>
#include<stdlib.h>
#define MAXV 100

// 边结点结构体
typedef struct EdgeNode
{
```

```

    int adjvex;        // 邻接顶点下标
    int weight;        // 边的权值
    struct EdgeNode *nextedge;
} EdgeNode;

// 顶点结构体
typedef struct VertexNode
{
    int data;
    EdgeNode *firstedge;
} VertexNode, AdjList[MAXV];

// 邻接表存储的图
typedef struct
{
    AdjList vertex;
    int n, e; // n:顶点总数 e:边总数
} GraphAdjList;

int inDegree[MAXV]; // 记录每个顶点入度
int topoSeq[MAXV];  // 存储拓扑排序序列
int ve[MAXV], vl[MAXV]; // ve:顶点最早发生时间 vl:顶点最迟发生时间

// 创建带权有向图邻接表
void CreateGraph(GraphAdjList *G)
{
    int i, u, v, w;
    EdgeNode *p;
    printf("请输入顶点数、边数: ");
    scanf("%d%d", &G->n, &G->e);

    // 初始化顶点数组与入度
    for (i = 0; i < G->n; i++)
    {
        G->vertex[i].data = i;
        G->vertex[i].firstedge = NULL;
        inDegree[i] = 0;
    }

    // 逐条输入有向边 u -> v, 权值w
    for (i = 0; i < G->e; i++)
    {
        printf("输入第%d条边 起点u 终点v 权值w: ", i + 1);
        scanf("%d%d%d", &u, &v, &w);
    }
}

```

```

        p = (EdgeNode *)malloc(sizeof(EdgeNode));
        p->adjvex = v;
        p->weight = w;
        p->nextedge = G->vertex[u].firstedge;
        G->vertex[u].firstedge = p;
        inDegree[v]++;
    }
}

// 打印邻接表结构
void PrintGraph(GraphAdjList *G)
{
    int i;
    EdgeNode *p;
    printf("\n图邻接表\n");
    for (i = 0; i < G->n; i++)
    {
        printf("顶点%d: ", G->vertex[i].data);
        p = G->vertex[i].firstedge;
        while (p != NULL)
        {
            printf("(邻接: %d, 权: %d) ", p->adjvex, p->weight);
            p = p->nextedge;
        }
        printf("\n");
    }
}

// 拓扑排序, 返回有效顶点个数 (有环则小于总顶点数)
int TopologicalSort(GraphAdjList *G)
{
    int stack[MAXV], top = -1;
    int i, cnt = 0, cur;
    EdgeNode *p;

    // 入度为0的顶点入栈
    for (i = 0; i < G->n; i++)
        if (inDegree[i] == 0)
            stack[++top] = i;

    while (top != -1)
    {
        cur = stack[top--];
        topoSeq[cnt++] = cur;
    }
}

```

```

        p = G->vertex[cur].firstedge;
        while (p != NULL)
        {
            int v = p->adjvex;
            inDegree[v]--;
            if (inDegree[v] == 0)
                stack[++top] = v;
            p = p->nextedge;
        }
    }
    return cnt;
}

```

```

// 计算各顶点最早发生时间 ve
void CalcVE(GraphAdjList *G, int cntTopo)
{
    int i, u, v;
    EdgeNode *p;
    for (i = 0; i < G->n; i++)
        ve[i] = 0;
    // 按拓扑正序遍历
    for (i = 0; i < cntTopo; i++)
    {
        u = topoSeq[i];
        p = G->vertex[u].firstedge;
        while (p != NULL)
        {
            v = p->adjvex;
            if (ve[v] < ve[u] + p->weight)
                ve[v] = ve[u] + p->weight;
            p = p->nextedge;
        }
    }
}

```

```

// 计算各顶点最迟发生时间 vl
void CalcVL(GraphAdjList *G, int cntTopo)
{
    int i, u, v;
    EdgeNode *p;

    //int T = ve[topoSeq[cntTopo - 1]];
    int T = 0;
    for (int i = 0; i < G->n; i++)

```

```

{
    if (ve[i] > T)
        T = ve[i];
}
for (i = 0; i < G->n; i++)
    vl[i] = T;

for (i = cntTopo - 1; i >= 0; i--)
{
    u = topoSeq[i];
    p = G->vertex[u].firstedge;

    while (p != NULL)
    {
        v = p->adjvex;
        if (vl[u] > vl[v] - p->weight)
            vl[u] = vl[v] - p->weight;

        p = p->nextedge;
    }
}

```

//输出关键活动、关键路径、总工期

```

int path[MAXV];
int pathLen = 0;

```

```

void PrintKeyPath(GraphAdjList *G)
{
    printf("\n关键活动（关键边）:\n");

    for (int u = 0; u < G->n; u++)
    {
        EdgeNode *p = G->vertex[u].firstedge;

        while (p != NULL)
        {
            int v = p->adjvex;

            // 关键边条件（核心）
            if (ve[u] + p->weight == ve[v])
            {

```

```

        printf("%d -> %d\n", u, v);
    }

    p = p->nextedge;
}
}

printf("工程总工期: %d\n", ve[topoSeq[G->n - 1]]);
}

int main()
{
    GraphAdjList G;
    int topoCount, i;
    CreateGraph(&G);
    PrintGraph(&G);

    // 执行拓扑排序
    topoCount = TopologicalSort(&G);
    if (topoCount < G.n)
    {
        printf("\n图存在环路，无法求解拓扑排序与关键路径! \n");
        return 0;
    }

    // 输出拓扑序列
    printf("\n拓扑排序序列\n");
    for (i = 0; i < topoCount; i++)
        printf("%d ", topoSeq[i]);
    printf("\n");

    // 计算ve、vl并输出关键路径
    CalcVE(&G, topoCount);
    CalcVL(&G, topoCount);
    PrintKeyPath(&G);

    return 0;
}

```

3.2.5 4. 调试分析

测试输入与完整运行输出

请输入顶点数、边数: 6 8
 输入第1条边 起点u 终点v 权值w: 0 1 3
 输入第2条边 起点u 终点v 权值w: 0 2 2

输入第3条边 起点u 终点v 权值w: 1 3 2
输入第4条边 起点u 终点v 权值w: 1 4 3
输入第5条边 起点u 终点v 权值w: 2 3 4
输入第6条边 起点u 终点v 权值w: 2 5 3
输入第7条边 起点u 终点v 权值w: 3 5 2
输入第8条边 起点u 终点v 权值w: 4 5 1

图邻接表

顶点0: (邻接: 2, 权: 2) (邻接: 1, 权: 3)
顶点1: (邻接: 4, 权: 3) (邻接: 3, 权: 2)
顶点2: (邻接: 5, 权: 3) (邻接: 3, 权: 4)
顶点3: (邻接: 5, 权: 2)
顶点4: (邻接: 5, 权: 1)
顶点5:

拓扑排序序列

0 1 4 2 3 5

关键活动 (关键边):

0 -> 2
0 -> 1
1 -> 4
2 -> 3
3 -> 5

工程总工期: 8

3.2.6 5. 小结

本实验实现有向图拓扑排序与关键路径算法, 能够判断有向图是否存在环, 熟练掌握 ve、vl 求解方法, 理解 AOE 网关键活动决定工程最短工期的原理。

3.3 3. 交通咨询系统

3.3.1 任务

设计一个简易交通咨询系统, 能让旅客咨询从一个城市到另一个城市之间的最短路径。功能要求: (1) 用图结构表示城市之间的交通联系, 图中的顶点为城市, 边上的权值为两城市之间的距离。建立交通网络图的存储结构, 并输出; (2) 求一个城市到其他城市的最短路径 (Dijkstra 算法), 输出路径顶点序列和总距离; (3) 求任一对城市之间的最短路径 (Floyd 算法), 输出路径顶点序列和总距离。

3.3.2 1. 需求分析

分析总结逻辑结构 采用邻接矩阵存储城市交通无向图, 顶点为城市、边权代表路程。

分析总结运算集合 运算包含交通图构建打印、Dijkstra 单源最短路径求解、Floyd 算法求解全部顶点对最短路径并输出路径长度。(2) 概要设计

3.3.3 2. 概要设计

存储结构设计 存储使用邻接矩阵保存路网，一维数组存城市名。Dijkstra 借助 dist、visited、path 数组松弛更新；Floyd 用 distF、pathF 二维数组，三层循环枚举中转点更新最短路径。

算法设计思路

3.3.4 3. 详细设计

源代码（带注释） 代码：

```
#include <stdio.h>
#include <string.h>

#define MAXV 100
#define INF 1000000

int G[MAXV][MAXV];
char city[MAXV][20];
int n, m;

//输出邻接矩阵
void printGraph()
{
    printf("\n交通网络邻接矩阵: \n\n");

    printf("%8s", "");
    for (int i = 0; i < n; i++)
        printf("%8s", city[i]);

    printf("\n");

    for (int i = 0; i < n; i++)
    {
        printf("%8s", city[i]);

        for (int j = 0; j < n; j++)
        {
            if (G[i][j] == INF)
                printf("%8s", "INF");
            else
                printf("%8d", G[i][j]);
        }

        printf("\n");
    }
}
```

```
}

//Dijkstra

void printDijkstraPath(int path[], int v)
{
    if (path[v] == -1)
    {
        printf("%s", city[v]);
        return;
    }

    printDijkstraPath(path, path[v]);
    printf(" -> %s", city[v]);
}

void Dijkstra(int src)
{
    int dist[MAXV];
    int visited[MAXV];
    int path[MAXV];

    for (int i = 0; i < n; i++)
    {
        dist[i] = G[src][i];
        visited[i] = 0;

        if (i != src && G[src][i] < INF)
            path[i] = src;
        else
            path[i] = -1;
    }

    dist[src] = 0;
    visited[src] = 1;

    for (int i = 1; i < n; i++)
    {
        int min = INF;
        int u = -1;

        for (int j = 0; j < n; j++)
        {
            if (!visited[j] && dist[j] < min)
```

```

        {
            min = dist[j];
            u = j;
        }
    }

    if (u == -1)
        break;

    visited[u] = 1;

    for (int v = 0; v < n; v++)
    {
        if (!visited[v] &&
            G[u][v] < INF &&
            dist[u] + G[u][v] < dist[v])
        {
            dist[v] = dist[u] + G[u][v];
            path[v] = u;
        }
    }
}

printf("Dijkstra最短路径 (起点: %s) \n", city[src]);

for (int i = 0; i < n; i++)
{
    if (i == src)
        continue;

    printf("\n%s -> %s\n", city[src], city[i]);

    if (dist[i] == INF)
    {
        printf("无路径\n");
        continue;
    }

    printf("路径: ");
    printDijkstraPath(path, i);

    printf("\n距离: %d\n", dist[i]);
}
}

```

```
//Floyd

int distF[MAXV][MAXV];
int pathF[MAXV][MAXV];

void Floyd()
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            distF[i][j] = G[i][j];

            if (i != j && G[i][j] < INF)
                pathF[i][j] = i;
            else
                pathF[i][j] = -1;
        }
    }

    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (distF[i][k] < INF &&
                    distF[k][j] < INF &&
                    distF[i][k] + distF[k][j] < distF[i][j])
                {
                    distF[i][j] = distF[i][k] + distF[k][j];
                    pathF[i][j] = pathF[k][j];
                }
            }
        }
    }
}

void printFloydPath(int start, int end)
{
    int stack[MAXV];
    int top = -1;
```

```

    int cur = end;

    while (cur != start)
    {
        stack[++top] = cur;
        cur = pathF[start][cur];
    }

    printf("%s", city[start]);

    while (top >= 0)
    {
        printf(" -> %s", city[stack[top--]]);
    }
}

void showFloyd()
{
    printf("Floyd任意两点最短路径\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            printf("\n%s -> %s\n", city[i], city[j]);

            if (distF[i][j] == INF)
            {
                printf("无路径\n");
                continue;
            }

            printf("路径: ");
            printFloydPath(i, j);

            printf("\n距离: %d\n", distF[i][j]);
        }
    }
}

int main()
{
    printf("请输入城市数: ");
    scanf("%d", &n);

```

```
printf("请输入城市名称: \n");

for (int i = 0; i < n; i++)
    scanf("%s", city[i]);

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (i == j)
            G[i][j] = 0;
        else
            G[i][j] = INF;
    }
}

printf("请输入道路数: ");
scanf("%d", &m);

printf("请输入道路信息(城市1 城市2 距离): \n");

for (int i = 0; i < m; i++)
{
    char a[20], b[20];
    int d;

    scanf("%s %s %d", a, b, &d);

    int u = -1, v = -1;

    for (int j = 0; j < n; j++)
    {
        if (strcmp(city[j], a) == 0)
            u = j;

        if (strcmp(city[j], b) == 0)
            v = j;
    }

    if (u == -1 || v == -1)
    {
        printf("城市不存在, 忽略该边! \n");
        continue;
    }
}
```

```

        }

        G[u][v] = d;
        G[v][u] = d;
    }

    printGraph();

    int src;

    printf("\n请输入Dijkstra起点编号(0~%d): ", n - 1);
    scanf("%d", &src);

    if (src < 0 || src >= n)
    {
        printf("起点编号错误! \n");
        return 0;
    }

    Dijkstra(src);

    Floyd();

    showFloyd();

    return 0;
}

```

3.3.5 4. 调试分析

测试输入与完整运行输出

请输入城市数: 4

请输入城市名称:

A

B

C

D

请输入道路数: 5

请输入道路信息(城市1 城市2 距离):

A B 2

A C 5

B C 1

B D 4

C D 3

交通网络邻接矩阵:

	A	B	C	D
A	0	2	5	INF
B	2	0	1	4
C	5	1	0	3
D	INF	4	3	0

请输入Dijkstra起点编号(0~3): 0

Dijkstra最短路径 (起点: A)

A -> B

路径: A -> B

距离: 2

A -> C

路径: A -> B -> C

距离: 3

A -> D

路径: A -> B -> D

距离: 6

Floyd任意两点最短路径

A -> B

路径: A -> B

距离: 2

A -> C

路径: A -> B -> C

距离: 3

A -> D

路径: A -> B -> D

距离: 6

B -> C

路径: B -> C

距离: 1

B -> D

路径: B -> D

距离: 4

C -> D

路径: C -> D

距离: 3

3.3.6 5. 小结

本交通咨询系统实现两种经典最短路径算法, Dijkstra 适合单源查询, Floyd 可一次性算出所有城市间路径, 掌握多源最松弛更新原理, 仅 Floyd 路径记录逻辑存在缺陷。

4 模块四

4.1 1. 基于线性表的查找（学生信息管理）

4.1.1 任务

建立学生信息表，实现顺序查找、折半查找。要求：（1）输入的学生数据包含学号、姓名、总成绩等多个数据项。（2）输出：查找成功输出找到的数据的所有数据项，查找失败也要给出相应提示信息。（3）分析总结算法各自的特点以及 ASL。

4.1.2 1. 需求分析

分析总结逻辑结构 以结构体线性表存储学生学号、姓名与成绩，提供顺序、折半两种学号查找功能，成功输出完整信息，失败给出提示。

分析总结运算集合

4.1.3 2. 概要设计

采用结构体数组作为有序线性表存储学生数据。顺序查找逐一遍历匹配学号；折半查找仅适用于有序表，通过不断缩小区间快速定位目标。

存储结构设计

算法设计思路

4.1.4 3. 详细设计

源代码（带注释） 代码：

```
#include <stdio.h>

#define MAXSIZE 100

// 学生结构体
typedef struct
{
    int id;           // 学号
    char name[20];    // 姓名
    float score;      // 总成绩
} Student;

// 顺序查找
int SequentialSearch(Student stu[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (stu[i].id == key)
            return i;
    }
}
```

```
    }
    return -1;
}

// 折半查找
int BinarySearch(Student stu[], int n, int key)
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (stu[mid].id == key)
            return mid;
        else if (stu[mid].id < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

// 输出学生信息
void PrintStudent(Student s)
{
    printf("学号: %d\n", s.id);
    printf("姓名: %s\n", s.name);
    printf("总成绩: %.2f\n", s.score);
}

int main()
{
    Student stu[MAXSIZE];
    int n;

    printf("请输入学生人数: ");
    scanf("%d", &n);

    printf("请按学号从小到大输入学生信息: \n");

    for (int i = 0; i < n; i++)
```

```
{
    printf("第%d个学生: \n", i + 1);

    printf("学号: ");
    scanf("%d", &stu[i].id);

    printf("姓名: ");
    scanf("%s", stu[i].name);

    printf("总成绩: ");
    scanf("%f", &stu[i].score);
}

int key;

printf("\n请输入要查找的学号: ");
scanf("%d", &key);

// 顺序查找
printf("\n顺序查找\n");

int pos = SequentialSearch(stu, n, key);

if (pos != -1)
{
    printf("查找成功! \n");
    PrintStudent(stu[pos]);
}
else
{
    printf("查找失败, 未找到该学生! \n");
}

// 折半查找
printf("\n折半查找\n");

pos = BinarySearch(stu, n, key);

if (pos != -1)
{
    printf("查找成功! \n");
    PrintStudent(stu[pos]);
}
else
```

```

{
    printf("查找失败，未找到该学生！\n");
}

return 0;
}

```

4.1.5 4. 调试分析

测试输入与完整运行输出

RESULT

4.1.6 5. 小结

本实验采用结构体建立学生信息表，以学号作为关键字，实现了顺序查找和折半查找两种查找方法，并统计了查找过程中的比较次数。通过实验发现，顺序查找的实现简单，不需要对数据进行排序，适用于无序表和各种存储结构，但随着数据规模的增加，其查找效率会明显下降。折半查找通过不断缩小查找区间来定位目标元素，平均查找长度远小于顺序查找，因此具有更高的查找效率，但必须保证查找表有序。

顺序查找成功时的平均查找长度（Average Search Length, ASL）为：顺序查找平均查找长度：

$$ASL = \frac{n+1}{2}$$

其时间复杂度为：

$$O(n)$$

折半查找要求查找表按关键字有序排列，其平均查找长度约为：

$$ASL \approx \log_2(n+1) - 1$$

时间复杂度为：

$$O(\log_2 n)$$

通过本次实验，加深了对查找表、关键字、平均查找长度（ASL）等概念的理解，掌握了顺序查找和折半查找的实现方法及适用条件，并认识到算法效率与数据组织方式之间的密切关系。在实际应用中，应根据数据是否有序以及查找效率要求选择合适的查找算法。对于无序表或数据经常发生变化的情况，顺序查找具有较好的适应性；而对于静态有序表，折半查找能够显著提高查找效率。

4.2 2. 基于树表的查找（文本词汇统计）

4.2.1 任务

读取一个文本文件，统计文件中每个单词的出现次数，并按字典序输出。要求：（1）使用二叉搜索树存储单词及其出现次数。（2）提供二叉搜索树上的插入、查找和中序遍历。（3）分析并总结二叉搜索树的特点以及 ASL。

4.2.2 1. 需求分析

分析总结逻辑结构 采用二叉搜索树链式存储单词与词频，属于非线性树形结构，左子树单词字典序小于根，右子树大于根，天然具备有序特性。

分析总结运算集合 包含文件字符读取、单词拆分小写转换、二叉树节点查找、插入计数、中序遍历输出、树内存释放等操作。

4.2.3 2. 概要设计

存储结构设计 二叉链表作为存储载体，每个节点存储单词字符串、出现次数，搭配左右孩子指针；文件文本临时用一维字符数组缓存单词。

算法设计思路 读取文本拆分标准化单词，递归查找 BST，存在则词频自增，不存在则新建节点插入；中序遍历按字典序输出全部单词，结束递归销毁整棵树释放内存。

4.2.4 3. 详细设计

源代码（带注释） 代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAXWORD 100

//二叉搜索树结点
typedef struct BSTNode
{
    char word[MAXWORD];
    int count;
    struct BSTNode *left;
    struct BSTNode *right;
} BSTNode;

typedef BSTNode* BSTree;

// 创建新结点
BSTNode* CreateNode(char *word)
{
    BSTNode *node = (BSTNode*)malloc(sizeof(BSTNode));

    strcpy(node->word, word);
    node->count = 1;
    node->left = NULL;
    node->right = NULL;

    return node;
}

// 查找
```

```
BSTNode* Search(BSTree root, char *word)
{
    if(root == NULL)
        return NULL;

    int cmp = strcmp(word, root->word);

    if(cmp == 0)
        return root;
    else if(cmp < 0)
        return Search(root->left, word);
    else
        return Search(root->right, word);
}

// 插入
BSTree Insert(BSTree root, char *word)
{
    if(root == NULL)
        return CreateNode(word);

    int cmp = strcmp(word, root->word);

    if(cmp == 0)
    {
        root->count++;
    }
    else if(cmp < 0)
    {
        root->left = Insert(root->left, word);
    }
    else
    {
        root->right = Insert(root->right, word);
    }

    return root;
}

// 中序遍历
void InOrder(BSTree root)
{
    if(root == NULL)
        return;
```

```
InOrder(root->left);

printf("%-15s %d\n", root->word, root->count);

InOrder(root->right);
}

// 将单词转换为小写
void ToLower(char *word)
{
    for(int i=0;word[i];i++)
        word[i]=tolower(word[i]);
}

// 从文件读取
void ReadFile(BSTree *root, char *filename)
{
    FILE *fp=fopen(filename,"r");

    if(fp==NULL)
    {
        printf("文件打开失败！\n");
        return;
    }

    char word[MAXWORD];
    int index=0;
    int ch;

    while((ch=fgetc(fp))!=EOF)
    {
        if(isalpha(ch))
        {
            word[index++]=tolower(ch);
        }
        else
        {
            if(index>0)
            {
                word[index]='\0';

                *root=Insert(*root,word);
            }
        }
    }
}
```

```

        index=0;
    }
}

// 文件最后一个单词
if(index>0)
{
    word[index]='\0';
    *root=Insert(*root,word);
}

fclose(fp);
}

// 释放树
void Destroy(BSTree root)
{
    if(root==NULL)
        return;

    Destroy(root->left);
    Destroy(root->right);

    free(root);
}

int main()
{
    BSTree root=NULL;

    ReadFile(&root,"text.txt");

    printf("单词统计结果: \n\n");
    printf("%-15s%s\n","Word","Count");
    InOrder(root);

    Destroy(root);

    return 0;
}

```

4.2.5 4. 调试分析

测试输入与完整运行输出

10

12 4 19 7 3 25 10 1 16 8

1 3 4 7 8 10 12 16 19 25

4.2.6 5. 小结

二叉搜索树的查找效率与树的形态有关。

当二叉搜索树较为平衡时，成功查找的平均查找长度（Average Search Length, ASL）约为：

$$ASL \approx \log_2 n$$

其平均时间复杂度为：

$$O(\log_2 n)$$

当二叉搜索树退化为单链表时，平均查找长度为：

$$ASL = \frac{n+1}{2}$$

其最坏时间复杂度为：

$$O(n)$$

二叉搜索树左小右大，中序遍历可实现字典序输出。其随机数据下查找插入效率高，有序数据会退化为单链表，有序场景可采用平衡树优化性能。

4.3 3. 基于哈希查找实现员工信息管理

4.3.1 任务

建立员工信息管理表，实现哈希查找。要求：（1）确定哈希函数以及解决冲突的方法。（2）员工信息包含工号、姓名、部门等数据项，实现员工的添加、查找、删除和遍历。（3）分析总结哈希查找的特点。

4.3.2 1. 需求分析

分析总结逻辑结构 采用一维数组存储哈希表，属于顺序线性结构，通过哈希映射将员工工号映射至数组下标存储记录。

分析总结运算集合 包含哈希映射、冲突探测、员工增删查、全表遍历、状态标记等基础操作。

4.3.3 2. 概要设计

存储结构设计 定长数组作为哈希表，结构体存员工信息，增设状态位区分空、占用、已删三种存储单元。

算法设计思路 除留余数法构造哈希函数，线性探测法处理冲突，标记删除避免查找断链，配套菜单实现交互。

4.3.4 3. 详细设计

源代码（带注释） 代码：

```
#include <stdio.h>
#include <string.h>

#define HASHSIZE 101

// 员工信息结构体
typedef struct
{
    int id;           // 工号
    char name[30];    // 姓名
    char dept[30];    // 部门
    int state;        // 0-空 1-已使用 2-已删除
} Employee;

// 哈希表
Employee HashTable[HASHSIZE];

// 初始化哈希表
void InitHashTable()
{
    int i;
    for(i = 0; i < HASHSIZE; i++)
    {
        HashTable[i].state = 0;
    }
}

// 哈希函数（除留余数法）
int Hash(int key)
{
    return key % HASHSIZE;
}

// 查找员工
int Search(int id)
{
    int addr = Hash(id);
    int start = addr;

    while(HashTable[addr].state != 0)
    {
        if(HashTable[addr].state == 1 &&
```

```
        HashTable[addr].id == id)
    {
        return addr;
    }

    addr = (addr + 1) % HASHSIZE;

    if(addr == start)
        break;
}

return -1;
}

// 添加员工
void Insert()
{
    Employee e;

    printf("请输入工号: ");
    scanf("%d", &e.id);

    if(Search(e.id) != -1)
    {
        printf("工号已存在! \n");
        return;
    }

    printf("请输入姓名: ");
    scanf("%s", e.name);

    printf("请输入部门: ");
    scanf("%s", e.dept);

    int addr = Hash(e.id);
    int start = addr;

    while(HashTable[addr].state == 1)
    {
        addr = (addr + 1) % HASHSIZE;

        if(addr == start)
        {
            printf("哈希表已满! \n");
```

```
        return;
    }
}

HashTable[addr] = e;
HashTable[addr].state = 1;

printf("添加成功! \n");
}

// 查找员工信息
void Find()
{
    int id;

    printf("请输入要查找的工号: ");
    scanf("%d", &id);

    int pos = Search(id);

    if(pos == -1)
    {
        printf("未找到该员工! \n");
    }
    else
    {
        printf("\n找到员工信息: \n");
        printf("工号: %d\n", HashTable[pos].id);
        printf("姓名: %s\n", HashTable[pos].name);
        printf("部门: %s\n", HashTable[pos].dept);
    }
}

// 删除员工
void Delete()
{
    int id;

    printf("请输入要删除的工号: ");
    scanf("%d", &id);

    int pos = Search(id);

    if(pos == -1)
```

```
{
    printf("员工不存在! \n");
    return;
}

HashTable[pos].state = 2;

printf("删除成功! \n");
}

// 遍历哈希表
void Traverse()
{
    int i;
    printf("哈希地址\t工号\t姓名\t部门\n");
    for(i = 0; i < HASHSIZE; i++)
    {
        if(HashTable[i].state == 1)
        {
            printf("%3d\t%d\t%s\t%s\n",
                i,
                HashTable[i].id,
                HashTable[i].name,
                HashTable[i].dept);
        }
    }
}

// 菜单
void Menu()
{
    printf("\n");
    printf("员工信息管理系统\n");
    printf("1. 添加员工\n");
    printf("2. 查找员工\n");
    printf("3. 删除员工\n");
    printf("4. 遍历员工\n");
    printf("0. 退出系统\n");
    printf("请选择: ");
}

int main()
{
    int choice;
```

```
InitHashTable();

while(1)
{
    Menu();
    scanf("%d", &choice);
    switch(choice)
    {
        case 1:
            Insert();
            break;

        case 2:
            Find();
            break;

        case 3:
            Delete();
            break;

        case 4:
            Traverse();
            break;

        case 0:
            printf("程序结束! \n");
            return 0;

        default:
            printf("输入错误, 请重新选择! \n");
    }
}

return 0;
}
```

4.3.5 4. 调试分析

测试输入与完整运行输出

员工信息管理系统

1. 添加员工
2. 查找员工
3. 删除员工
4. 遍历员工

0. 退出系统
请选择：1
请输入工号：202601
请输入姓名：LiMing
请输入部门：研发部
添加成功！

员工信息管理系统

- 1. 添加员工
- 2. 查找员工
- 3. 删除员工
- 4. 遍历员工
- 0. 退出系统

请选择：1
请输入工号：202602
请输入姓名：ZhangHua
请输入部门：市场部
添加成功！

员工信息管理系统

- 1. 添加员工
- 2. 查找员工
- 3. 删除员工
- 4. 遍历员工
- 0. 退出系统

请选择：4

哈希地址	工号	姓名	部门
96	202601	LiMing	研发部
97	202602	ZhangHua	市场部

员工信息管理系统

- 1. 添加员工
- 2. 查找员工
- 3. 删除员工
- 4. 遍历员工
- 0. 退出系统

请选择：2
请输入要查找的工号：202601
找到员工信息：
工号：202601
姓名：LiMing
部门：研发部

员工信息管理系统

- 1. 添加员工
- 2. 查找员工
- 3. 删除员工
- 4. 遍历员工

0. 退出系统
请选择：3
请输入要删除的工号：202601
删除成功！
员工信息管理系统
1. 添加员工
2. 查找员工
3. 删除员工
4. 遍历员工
0. 退出系统
请选择：4
哈希地址 工号 姓名 部门
97 202602 ZhangHua 市场部
员工信息管理系统
1. 添加员工
2. 查找员工
3. 删除员工
4. 遍历员工
0. 退出系统
请选择：2
请输入要查找的工号：202601
未找到该员工！
员工信息管理系统
1. 添加员工
2. 查找员工
3. 删除员工
4. 遍历员工
0. 退出系统
请选择：1
请输入工号：202602
工号已存在！
员工信息管理系统
1. 添加员工
2. 查找员工
3. 删除员工
4. 遍历员工
0. 退出系统
请选择：WangWu
请输入工号：请输入姓名：请输入部门：人事部
添加成功！
员工信息管理系统
1. 添加员工
2. 查找员工
3. 删除员工

4. 遍历员工
0. 退出系统
请选择：0
程序结束！

4.3.6 5. 小结

本程序基于哈希表实现员工管理，哈希查找平均时间复杂度接近 $O(1)$ ，查找效率极高；线性探测易产生堆积，删除需标记不可直接置空，适合高频查询场景。

5 模块五

5.1 1. 各种排序实现对比

5.1.1 任务

利用相关排序算法，将随机生成的一组整数（ $20 \leq \text{个数} \leq 100$ ）按递增的顺序排好。要求：（1）实现直接插入排序、希尔排序，冒泡排序、快速排序，简单选择排序、堆排序等算法。（2）输出的形式：数字大小逐个递增的数列。（3）比较不同排序算法的时间复杂度、空间复杂度，优缺点，以及适用场合。

5.1.2 1. 需求分析

分析总结逻辑结构 采用一维顺序数组存储待排序整数，属于连续线性顺序存储结构，元素下标对应数字位置，便于批量遍历与移位交换。

分析总结运算集合 核心操作包含元素比较、相邻 / 跨位置交换、数组移位、分组插入、堆调整、递归划分等基础运算。

5.1.3 2. 概要设计

存储结构设计 静态一维数组存放待排序数字，下标从 1 开始适配各类排序算法边界逻辑，统一数组接口复用全部排序函数。

算法设计思路 分别实现插入类、交换类、选择类共六种经典排序；统一递增排序标准，完成多算法横向对比，分析复杂度与适用场景。

5.1.4 3. 详细设计

源代码（带注释） 代码：

```
//直接插入排序
void insert_sort()
{
    for(int i = 2; i <= n; i++)
    {
        int key = a[i];
        int j = i - 1;
        while(j >= 1 && a[j] > key)
        {
```

```
    a[j + 1] = a[j];
    j--;
}
a[j + 1] = key;
}
}
//希尔排序
void shell_sort()
{
    for(int gap = n / 2; gap > 0; gap /= 2)
    {
        for(int i = gap + 1; i <= n; i++)
        {
            int tmp = a[i];
            int j = i - gap;

            while(j > 0 && a[j] > tmp)
            {
                a[j + gap] = a[j];
                j -= gap;
            }

            a[j + gap] = tmp;
        }
    }
}
//冒泡排序
void swap(int* a, int * b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

void bubble_sort()
{
    for(int i = 1; i < n; i++)
    {
        int flag = 0;
        for(int j = 1; j <= n - i; j++)
        {
            if(a[j] > a[j + 1])
            {
                swap(&a[j], &a[j + 1]);
            }
        }
    }
}
```

```

        flag = 1;
    }
}
if(!flag) break;
}
}
//快速排序
void swap(int* a, int * b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int get_radom_index(int left, int right)
{
    return (rand() % (right - left + 1)) + left;
}

void quick_sort(int left, int right)
{
    if(left >= right) return;
    int ele = a[get_radom_index(left, right)];
    int l = left - 1, i = left, r = right + 1;
    while(i < r)
    {
        if(ele > a[i]) swap(&a[++l], &a[i++]);
        else if(ele == a[i]) i++;
        else swap(&a[--r], &a[i]);
    }
    quick_sort(left, l);
    quick_sort(r, right);
}
//堆排序
void swap(int* a, int * b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

void down(int parent, int len)
{

```

```

int child = parent * 2;
while(child <= len)
{
    if(child + 1 <= len && a[child + 1] > a[child])
        child++;
    if(a[parent] > a[child])
        break;
    swap(&a[parent], &a[child]);
    parent = child;
    child = parent * 2;
}
}

void heap_sort()
{
    //建堆
    for(int i = n / 2; i >= 1; i--)
    {
        down(i, n);
    }
    //排序
    for(int i = n; i >= 1; i--)
    {
        swap(&a[1], &a[i]);
        down(1, i - 1);
    }
}

```

5.1.5 4. 调试分析

测试输入与完整运行输出

```

10
12 4 19 7 3 25 10 1 16 8
1 3 4 7 8 10 12 16 19 25

```

5.1.6 5. 小结

- 直接插入排序 $O(n^2)$
- 希尔排序最优 $O(n)$ 最差 $O(n^2)$
- 冒泡排序 $O(n^2)$
- 快速排序 $O(n\log n)$ 最差 $O(n^2)$
- 堆排序 $O(n\log n)$